LAPORAN PROYEK AKHIR
DASAR-DASAR PEMROGRAMAN
ADMINISTRASI TRANSAKSI KLINIK



Disusun Oleh:

KELOMPOK 3

Muhammad Atha Andhika	2609116065
Nabilah Rahmadhani	2609116041
Mario Bawa Welan	2609116054
Annisa Maulida Rizky	2609116061

Asisten Laboratorium:

Hanif Amelia Putri	**Muhammad Davi Aditya Pratama**
2509116075	**2509116070**

PROGRAM STUDI SISTEM INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS MULAWARMAN

2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga kami dapat menyelesaikan laporan proyek pemrograman yang berjudul **"Administrasi Transaksi Klinik"** dengan baik. Laporan ini disusun untuk memenuhi tugas mata kuliah Dasar Pemrograman sekaligus sebagai dokumentasi dari proses pembuatan program menggunakan bahasa pemrograman Python.

Program Administrasi Transaksi Klinik dibuat sebagai penerapan konsep dasar pemrograman dalam pengelolaan administrasi transaksi secara sederhana. Program ini memiliki beberapa Fitur, seperti pengelolaan akun pengguna dan layanan, pengisian saldo, pembayaran layanan, pencatatan riwayat transaksi, pencetakan invoice, serta penyajian laporan ringkasan transaksi.

Dalam proses penyusunan laporan dan pembuatan program ini, kami memperoleh kesempatan untuk meningkatkan pemahaman serta keterampilan dalam menerapkan konsep pemrograman untuk menyelesaikan permasalahan sederhana. Kami berharap laporan ini dapat memberikan gambaran yang jelas mengenai proses pembuatan dan Fungsi program yang dikembangkan.

Kami menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kami mengharapkan kritik dan saran yang membangun sebagai bahan evaluasi untuk perbaikan di masa mendatang. Semoga laporan ini dapat bermanFaat bagi kami sebagai penyusun maupun bagi pembaca.

Oktober, 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR ..... **ii**

DAFTAR ISI ..... **iii**

DAFTAR GAMBAR ..... **v**

DAFTAR TABEL ..... **viii**

DAFTAR LAMPIRAN ..... **ix**

BAB I PENDAHULUAN **1**

1.1 Deskripsi Masalah 1

1.2 Rumusan Masalah 2

1.3 Batasan Masalah 2

1.4 Tujuan 3

1.5 Manfaat 3

BAB II PERANCANGAN **4**

2.1 Analisis Program 4

2.2 Flowchart Menu Utama 6

2.3 Flowchart Menu User 9

2.4 Flowchart Menu Admin 12

BAB III HASIL DAN PEMBAHASAN **16**

3.1 Implementasi Program 16

3.2 Alur Program 17

3.2.1. Import library 17

3.2.2. Pengaturan lokasi File csv 18

3.2.3. Menentukan Struktur Kolom Data 19

3.2.4. Fungsi pendukung program 20

3.2.5. Fungsi Tampilan Judul dan Tabel 23

3.2.6. Fungsi membaca dan menyimpan file CSV 25

3.2.7. Pengolahan data akun dan data awal 27

3.2.8. Fungsi Pencarian Data Pengguna dan Layanan 29

3.2.9. Registrasi Pengguna 30

3.2.10. Login Pengguna 33

3.2.11. Pengelolaan daftar layanan klinik 34

3.2.12. Pencatatan Transaksi, Pembuatan ID Transaksi dan Pembuatan Invoice.....40
3.2.13. Top-up Saldo.....42
3.2.14. Pembayaran Layanan Klinik.....44
3.2.15. Riwayat Transaksi.....46
3.2.16. Pengelolaan Pengguna oleh Admin.....48
3.2.17. Laporan Transaksi untuk Admin.....50
3.2.18. Menu Admin.....51
3.2.19. Menu User.....53
3.2.20. Menu Autentikasi.....55
3.2.21. Fungsi Utama Program (Main).....57
3.2.22. Menjalankan Program.....59
3.3 Source Code.....61
BAB IV PENUTUP.....84
4.1 Kesimpulan.....84
4.2 Saran.....84
LAMPIRAN.....85

DAFTAR GAMBAR

Gambar 2. 1. Flowchart Menu Utama 6
Gambar 2. 2. Flowchart Menu User 9
Gambar 2. 3. Flowchart Menu Admin 12
Gambar 3. 1. Import Library 17
Gambar 3. 2. Tempat Folder 18
Gambar 3. 3. Lokasi File CSV 18
Gambar 3. 4. Lokasi File dan Role 18
Gambar 3. 5. Kolom Users 19
Gambar 3. 6. Kolom Layanan 19
Gambar 3. 7. Kolom Transaksi 20
Gambar 3. 8. Fungsi Membersihkan Layar 21
Gambar 3. 9. Fungsi Jeda 21
Gambar 3. 10. Fungsi input teks dan validasi 21
Gambar 3. 11. Fungsi input password 22
Gambar 3. 12. Fungsi input angka 22
Gambar 3. 13. Fungsi konfirmasi 23
Gambar 3. 14. Fungsi format rupiah 23
Gambar 3. 15. Fungsi menampilkan judul 23
Gambar 3. 16. Fungsi menampilkan table 24
Gambar 3. 17. Fungsi membaca file CSV 25
Gambar 3. 18. Fungsi menyimpan file CSV 26
Gambar 3. 19. Fungsi membaca data layanan 26
Gambar 3. 20. Fungsi menyimpan data layanan 27
Gambar 3. 21. Fungsi kunci pengurutan ID layanan 27
Gambar 3. 22. Fungsi hash password 27
Gambar 3. 23. Fungsi insialisasi data 27
Gambar 3. 24. Membaca data user 28
Gambar 3. 25. Data daftar layanan 28
Gambar 3. 26. Memeriksa file layanna yang tersedia 29
Gambar 3. 27. Fungsi mencari pengguna 29
Gambar 3. 28. Fungsi mencari layanan 30

Gambar 3. 29. Fungsi registrasi pengguna 30
Gambar 3. 30. Membuat username dan password 31
Gambar 3. 31. Masukkan nama lengkap dan buat akun baru 32
Gambar 3. 32. Login pengguna 33
Gambar 3. 33. Percobaan login 33
Gambar 3. 34. Menampilkan daftar layanan 34
Gambar 3. 35. Menambilkan tabel layanan 35
Gambar 3. 36. Menambahkan layanan 35
Gambar 3. 37. Lanjutan menambah layanan 36
Gambar 3. 38. Mengubah layanan 38
Gambar 3. 39. Menghapus layanan 39
Gambar 3. 40. Pembuatan ID transaksi 40
Gambar 3. 41. Memasukkan ID Transaksi 40
Gambar 3. 42. Menghitung total biaya layanan 41
Gambar 3. 43. Membuat invoice transaksi 42
Gambar 3. 44. Memasukkan nominal top up 42
Gambar 3. 45. Validasi nominal top up 43
Gambar 3. 46. Memperbarui saldo pengguna 43
Gambar 3. 47. Memilih layanan yang akan dibayar 44
Gambar 3. 48. Memeriksa kecukupan saldo 45
Gambar 3. 49. Memproses pembayaran dan memperbarui saldo 45
Gambar 3. 50. Menampilkan riwayat traksaksi pengguna 46
Gambar 3. 51. Menampilkan detail transaksi 47
Gambar 3. 52. Menyimpan data riwayat transaksi 48
Gambar 3. 53. Menampilkan daftar pengguna 48
Gambar 3. 54. Melihat informasi pengguna 48
Gambar 3. 55. Mengelola data pengguna 49
Gambar 3. 56. Menampilkan data transaksi 50
Gambar 3. 57. Menghitung total transaksi 50
Gambar 3. 58. Menyajikan laporan transaksi ..... **Error! Bookmark not defined.**
Gambar 3. 59. Menampilkan menu admin 51
Gambar 3. 60. Memilih menu administrasi 52
Gambar 3. 61. Menjalankan fungsi admin 52
Gambar 3. 62. Menampilkan menu pengguna 53

Gambar 3. 63. Memilih fitur layanan 54
Gambar 3. 64. Menjalankan fungsi pengguna 54
Gambar 3. 65. Proses login pengguna 55
Gambar 3. 66. Validasi username dan password 56
Gambar 3. 67. Pembagian hak akses pengguna 56
Gambar 3. 68. Memulai program 57
Gambar 3. 69. Menampilkan menu utama 57
Gambar 3. 70. Mengatur alur jalannya program 58
Gambar 3. 71. Persiapan file data 59
Gambar 3. 72. Proses eksekusi program 59
Gambar 3. 73. Pengujian fungsi program 60

DAFTAR TABEL

Tabel 2. 1. Library yang digunakan dan fungsinya.....4
Tabel 3. 1. Source Code……61

DAFTAR LAMPIRAN

Lampiran 1. Tabel Kontribusi 85

BAB I
PENDAHULUAN

1.1 Deskripsi Masalah

Perkembangan teknologi informasi memungkinkan pengelolaan data dilakukan secara lebih terstruktur melalui program komputer. Dalam administrasi transaksi klinik, data layanan, akun pengguna, saldo, dan riwayat transaksi perlu dikelola dengan baik agar informasi mudah ditelusuri dan laporan dapat disusun berdasarkan data yang tercatat.

Dalam kegiatan administrasi, data layanan dan transaksi perlu dikelola dengan baik agar informasi dapat ditemukan dan diperiksa dengan mudah. Jika pencatatan data tidak dilakukan secara teratur, pencarian informasi, pengecekan riwayat transaksi, pemantauan perubahan saldo, dan pembuatan laporan dapat menjadi lebih sulit. Oleh sebab itu, diperlukan sebuah program sederhana yang mampu mengelola data layanan, akun pengguna, saldo, dan transaksi dalam satu sistem.

Untuk mendukung kebutuhan tersebut, dibuat program **Administrasi Transaksi Klinik berbasis Python** yang menyediakan fitur login dan registrasi pengguna, pengelolaan layanan oleh admin, pengisian saldo, pembayaran layanan, riwayat transaksi, pencetakan invoice, serta laporan ringkasan transaksi. Program menggunakan berkas CSV untuk menyimpan data pengguna, layanan, dan transaksi.

Selain itu, program ini menyediakan fitur riwayat transaksi dan laporan ringkasan yang dapat digunakan untuk melihat kembali aktivitas transaksi yang telah tercatat. Melalui fitur tersebut, informasi pembayaran dan catatan transaksi dapat diperiksa dengan lebih mudah sehingga pengelolaan data menjadi lebih teratur.

Melalui pembuatan program ini, konsep dasar pemrograman Python diterapkan dalam pengelolaan data, pembagian hak akses, perhitungan saldo, pencatatan transaksi, dan penyusunan laporan. Program ini dibatasi sebagai simulasi administrasi transaksi dan tidak mencakup rekam medis maupun pelayanan medis.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam pembuatan program Administrasi Transaksi Klinik berbasis Python adalah sebagai berikut.

1. Bagaimana merancang dan membangun program administrasi transaksi klinik menggunakan bahasa pemrograman Python?
2. Bagaimana menerapkan pengelolaan layanan, akun pengguna, saldo, dan pembayaran dalam satu program?
3. Bagaimana menerapkan penyimpanan data dan pencatatan riwayat transaksi menggunakan berkas CSV?
4. Bagaimana menyediakan Fitur invoice dan laporan transaksi untuk mendukung pencatatan serta pemantauan transaksi?

1.3 Batasan Masalah

Agar pembahasan lebih terarah dan sesuai dengan program yang dikembangkan, batasan masalah ditetapkan sebagai berikut.

1. Program dibuat menggunakan bahasa pemrograman Python dan dijalankan melalui antarmuka terminal.
2. Sistem memiliki dua jenis pengguna, yaitu admin dan user, dengan hak akses yang berbeda.
3. Fitur program mencakup registrasi dan login, pengelolaan layanan dan akun pengguna, pengisian saldo, serta pembayaran layanan menggunakan saldo internal.
4. Program menyediakan fitur riwayat transaksi, pencetakan invoice, dan laporan ringkasan transaksi.
5. Data pengguna, layanan, dan transaksi disimpan menggunakan berkas CSV.
6. Program merupakan simulasi administrasi transaksi klinik dan tidak mencakup pengelolaan rekam medis, diagnosis, resep obat, penjadwalan konsultasi, maupun pembayaran melalui sistem eksternal.

1.4 Tujuan

Tujuan dari pembuatan program Administrasi Transaksi Klinik ini adalah sebagai berikut.

1. Merancang dan membangun program administrasi transaksi klinik menggunakan bahasa pemrograman Python dengan antarmuka berbasis terminal.
2. Menerapkan pengelolaan layanan, akun pengguna, saldo, dan pembayaran layanan dalam satu program dengan pembagian hak akses admin dan user.
3. Menerapkan penyimpanan data pengguna, layanan, dan transaksi menggunakan berkas CSV agar data dapat disimpan dan digunakan kembali.
4. Menyediakan fitur riwayat transaksi, pencetakan invoice, dan laporan ringkasan transaksi untuk membantu proses pencatatan serta pemantauan transaksi klinik.

1.5 Manfaat

Pembuatan program Administrasi Transaksi Klinik ini diharapkan dapat memberikan manfaat sebagai berikut.

1. Bagi Pengembang

   Pembuatan program Administrasi Transaksi Klinik dapat menambah pengetahuan dan keterampilan pengembang dalam menggunakan bahasa pemrograman Python. Selain itu, proyek ini menjadi sarana untuk menerapkan konsep dasar pemrograman, pengelolaan data menggunakan berkas CSV, serta penerapan sistem login dan pencatatan transaksi.

2. Bagi Pengguna

   Program ini dapat memudahkan pengguna dalam melihat layanan yang tersedia, melakukan pengisian saldo, membayar layanan menggunakan saldo internal, melihat riwayat transaksi, serta memperoleh invoice sebagai rincian transaksi yang telah dilakukan.

3. Bagi Pembelajaran

   Program ini dapat menjadi contoh penerapan ilmu dasar pemrograman dalam membuat aplikasi sederhana untuk kebutuhan administrasi.

BAB II
PERANCANGAN

2.1 Analisis Program

Dalam pembuatan program Administrasi Transaksi Klinik, diperlukan bahasa pemrograman Python sebagai dasar pengembangan sistem. Program ini juga menggunakan beberapa library untuk mendukung proses pengelolaan data, keamanan kata sandi, pencatatan waktu, dan tampilan antarmuka berbasis terminal. Selain itu, diperlukan file CSV untuk menyimpan data pengguna, daftar layanan klinik, serta riwayat transaksi agar data tetap tersimpan meskipun program ditutup dan dijalankan kembali.

Program menggunakan tiga file CSV, yaitu users.csv, layanan.csv, dan transaksi.csv. File users.csv menyimpan informasi akun pengguna, kata sandi dalam bentuk hash, role, saldo, dan nama. File layanan.csv menyimpan ID layanan, nama layanan, dan harga. Sementara itu, file transaksi.csv menyimpan riwayat transaksi, seperti waktu transaksi, jenis transaksi, layanan yang digunakan, nominal pembayaran, saldo awal, saldo akhir, dan status transaksi. File CSV juga memungkinkan data disimpan dan dibaca kembali.

Program ini menggunakan beberapa library bawaan Python dan library tambahan, yaitu:

Tabel 2. 1. Library yang digunakan dan fungsinya

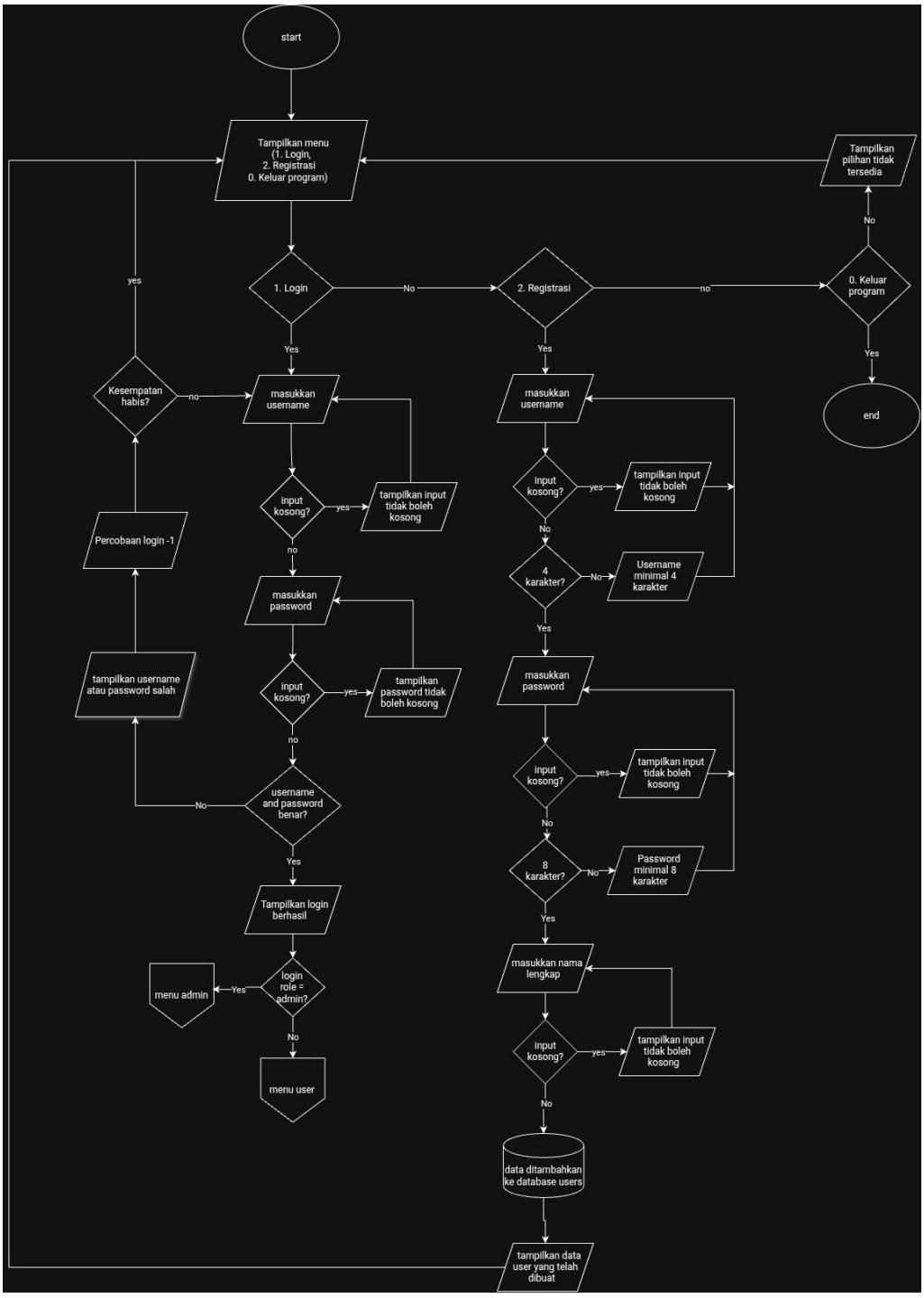
Library	Fungsi
CSV	Digunakan untuk membaca dan menyimpan data dalam format CSV. Library ini membantu program mengelola data pengguna, daftar layanan klinik, dan riwayat transaksi secara permanen.
OS	Digunakan untuk mengelola lokasi dan keberadaan file, menggabungkan alamat file, serta membersihkan layar terminal sesuai dengan sistem operasi yang digunakan.
Time	Digunakan untuk memberikan jeda waktu, misalnya setelah proses login berhasil agar pesan selamat datang dapat ditampilkan sebelum program melanjutkan proses berikutnya.

Library	Fungsi
Random	Digunakan untuk menghasilkan angka acak dalam pembentukan ID transaksi.
Hashlib	Digunakan untuk mengubah kata sandi menjadi bentuk hash menggunakan algoritma SHA-256 sebelum disimpan ke file CSV. Dengan demikian, kata sandi tidak disimpan dalam bentuk teks asli.
Datetime	Digunakan untuk mencatat tanggal dan waktu transaksi serta membantu membentuk ID transaksi berdasarkan tanggal.
PrettyTable	Library tambahan yang digunakan untuk menampilkan data dalam bentuk tabel agar informasi pengguna, layanan klinik, dan riwayat transaksi lebih rapi serta mudah dibaca.
Pwinput	Library tambahan yang digunakan untuk menerima input kata sandi tanpa menampilkan karakter kata sandi secara langsung di terminal.

Program ini juga menggunakan fungsi (def) untuk memisahkan setiap proses agar kode lebih terstruktur dan mudah dikelola. Contohnya, fungsi login digunakan untuk proses masuk ke akun, registrasi untuk pembuatan akun baru, topup_saldo untuk menambah saldo, dan bayar_layanan untuk memproses pembayaran layanan klinik. Program juga menggunakan perulangan while untuk menampilkan menu secara berulang serta percabangan if, elif, dan else untuk menentukan tindakan berdasarkan pilihan pengguna dan memvalidasi data yang dimasukkan.

Validasi input juga diperlukan di program ini untuk mengurangi kesalahan saat program digunakan. Program memeriksa agar input tidak kosong, username memenuhi ketentuan, password memiliki panjang minimal, harga dan nominal berupa angka yang sesuai, serta pilihan menu tersedia. Program juga menerapkan dua role pengguna, yaitu admin dan user. Admin memiliki akses untuk mengelola layanan, mengelola pengguna, melihat seluruh transaksi, dan melihat laporan transaksi. Sementara itu, user dapat melihat layanan, melakukan top-up saldo, membayar layanan, dan melihat riwayat transaksinya sendiri.

2.2 Flowchart Menu Utama



Gambar 2. 1. Flowchart Menu Utama

1. Mulai (Start)

Program dimulai ketika sistem Administrasi Transaksi Klinik dijalankan. Tahap ini merupakan awal dari keseluruhan proses program.

2. Menampilkan Menu Utama

Sistem akan menampilkan 3 pilihan utama kepada pengguna:

a. Login
b. Registrasi
c. Keluar

Pengguna harus memilih salah satu opsi untuk melanjutkan proses.

3. Proses Login

Jika pengguna memilih opsi login, sistem akan menjalankan Langkah-langkah sebagai berikut:

a. Memasukkan Username

   Pengguna diminta memasukkan username yang telah terdaftar. Sistem memeriksa apakah input kosong. Jika kosong, sistem menampilkan pesan bahwa username tidak boleh kosong dan meminta pengguna memasukkan kembali.

b. Memasukkan Password

   Setelah username terisi, pengguna diminta memasukkan password. Sistem kembali memeriksa apakah input kosong. Jika kosong, sistem menampilkan pesan kesalahan dan meminta pengguna memasukkan password kembali.

c. Verifikasi Username dan Password

   Sistem memeriksa kesesuaian username dan password dengan data akun yang tersimpan. Jika data tidak sesuai, sistem menampilkan pesan bahwa username atau password salah, kemudian menghitung jumlah percobaan login. Pengguna diberikan kesempatan untuk mencoba kembali. Apabila kesempatan habis, sistem mengarahkan pengguna kembali ke menu utama. Jika data benar, sistem menampilkan pesan bahwa login berhasil.

d. Pemeriksaan Hak Akses

   Setelah login berhasil, sistem memeriksa role atau hak akses akun pengguna. Jika akun memiliki role admin, pengguna diarahkan ke menu admin. Jika akun memiliki role user, pengguna diarahkan ke menu user.

4. Proses Registrasi

Jika pengguna memilih menu registrasi, sistem menjalankan proses pembuatan akun baru melalui tahapan berikut:

a. Memasukkan Username

Pengguna diminta memasukkan username yang akan digunakan untuk akun baru. Sistem memeriksa apakah input kosong dan memastikan username memiliki minimal 4 karakter. Jika tidak memenuhi ketentuan, sistem menampilkan pesan kesalahan dan meminta pengguna memasukkan kembali username.

b. Memasukkan Password

Setelah username memenuhi ketentuan, pengguna diminta memasukkan password. Sistem memeriksa apakah input kosong dan memastikan password memiliki minimal 8 karakter. Jika tidak sesuai, sistem menampilkan pesan kesalahan dan meminta pengguna memasukkan kembali password.

c. Memasukkan Nama Lengkap

Pengguna diminta memasukkan nama lengkap sebagai identitas akun. Sistem memeriksa apakah input kosong. Jika kosong, sistem menampilkan pesan bahwa nama lengkap tidak boleh kosong dan meminta pengguna mengulang input.

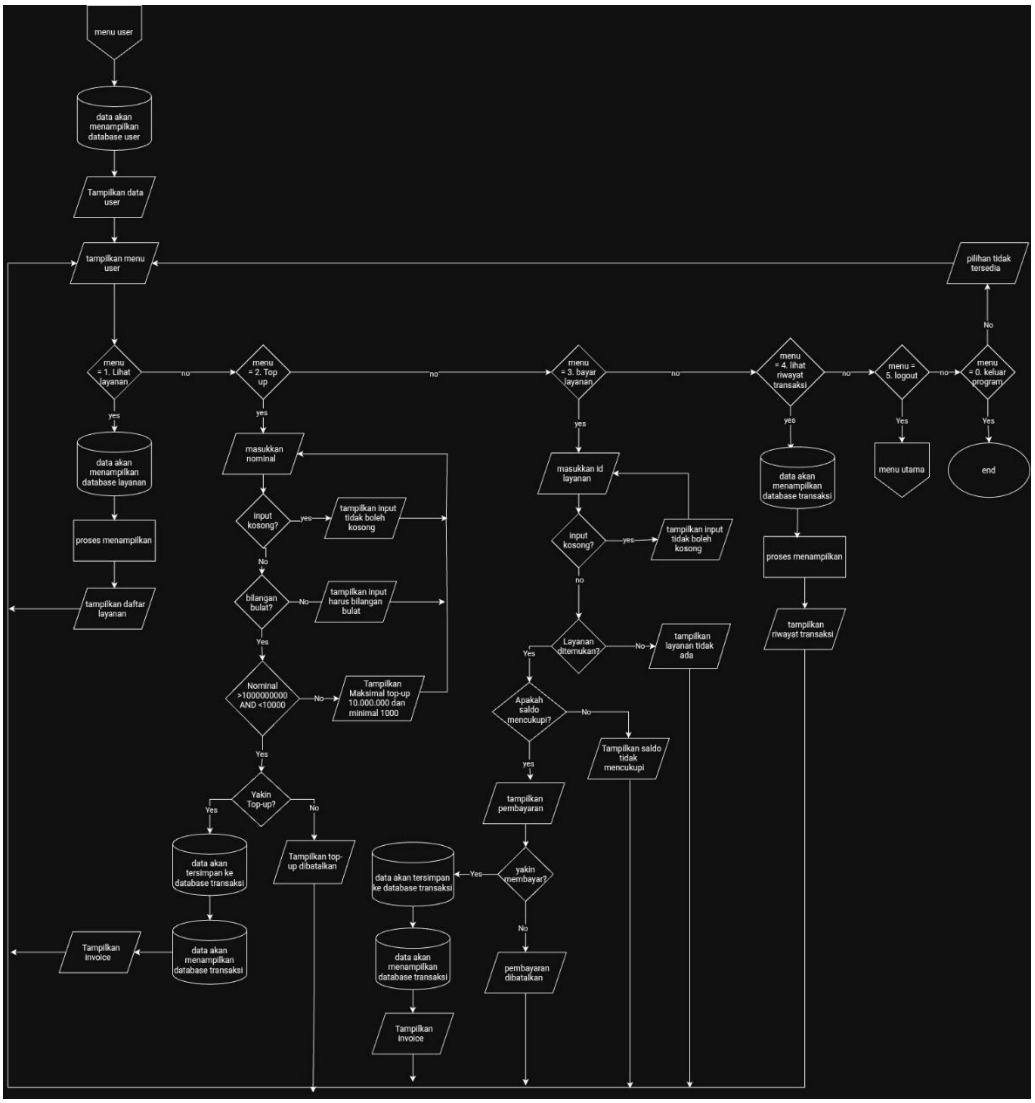
d. Menyimpan Data Registrasi

Jika seluruh data telah memenuhi ketentuan, sistem menyimpan informasi akun baru ke dalam database users. Setelah itu, sistem menampilkan data pengguna yang telah berhasil dibuat dan mengarahkan pengguna kembali ke menu utama.

5. Keluar Program

Jika pengguna memilih menu keluar, sistem mengakhiri seluruh proses program. Tahap ini menandai berakhirnya penggunaan sistem.

2.3 Flowchart Menu User



Gambar 2. 2. Flowchart Menu User

1. Menampilkan Data User

Setelah pengguna berhasil login sebagai user, sistem mengambil data akun dari database users, kemudian menampilkan informasi pengguna yang sedang masuk ke dalam sistem.

2. Menampilkan Menu User

Sistem menampilkan enam pilihan menu yang dapat digunakan oleh pengguna, yaitu:

a. Lihat Layanan

b. Top Up

c. Bayar Layanan

d. Lihat Riwayat Transaksi

e. Logout

f. Keluar Program

Pengguna memilih salah satu menu sesuai kebutuhan. Jika pilihan tidak tersedia, sistem menampilkan pesan kesalahan dan mengarahkan pengguna kembali ke menu user.

3. Menu Lihat Layanan

Jika pengguna memilih menu lihat layanan, sistem mengambil data dari database layanan, kemudian memproses dan menampilkan daftar layanan klinik yang tersedia. Setelah daftar layanan ditampilkan, pengguna diarahkan kembali ke menu user.

4. Menu Top Up

Jika pengguna memilih menu top up, sistem meminta pengguna memasukkan nominal saldo yang ingin ditambahkan. Sistem memeriksa apakah input kosong, memastikan nominal berupa bilangan bulat, serta memvalidasi batas nominal top up, yaitu minimal Rp1.000 dan maksimal Rp10.000.000. Jika input tidak sesuai, sistem menampilkan pesan kesalahan dan meminta pengguna memasukkan nominal kembali. Jika nominal valid, sistem menampilkan konfirmasi top up. Apabila pengguna menyetujui, data top up disimpan ke database transaksi, saldo diperbarui, dan sistem menampilkan invoice. Jika dibatalkan, proses top up dihentikan dan pengguna kembali ke menu user.

5. Menu Bayar Layanan

Jika pengguna memilih menu bayar layanan, sistem meminta pengguna memasukkan ID layanan yang ingin dibayar. Sistem memeriksa apakah input kosong dan memastikan layanan tersebut tersedia. Jika layanan tidak ditemukan, sistem menampilkan pesan bahwa layanan tidak tersedia. Jika ditemukan, sistem memeriksa apakah saldo pengguna mencukupi. Apabila saldo tidak mencukupi, sistem menampilkan pemberitahuan dan membatalkan proses pembayaran. Jika saldo mencukupi, sistem menampilkan informasi pembayaran dan meminta konfirmasi kepada pengguna. Jika pembayaran disetujui, transaksi disimpan ke database transaksi, saldo pengguna diperbarui, dan invoice ditampilkan. Jika

pembayaran dibatalkan, sistem menghentikan proses tanpa melanjutkan transaksi.

6. **Menu Lihat Riwayat Transaksi**

   Jika pengguna memilih menu lihat riwayat transaksi, sistem mengambil data transaksi dari database transaksi, kemudian memproses dan menampilkan riwayat transaksi pengguna. Setelah informasi ditampilkan, pengguna diarahkan kembali ke menu user.

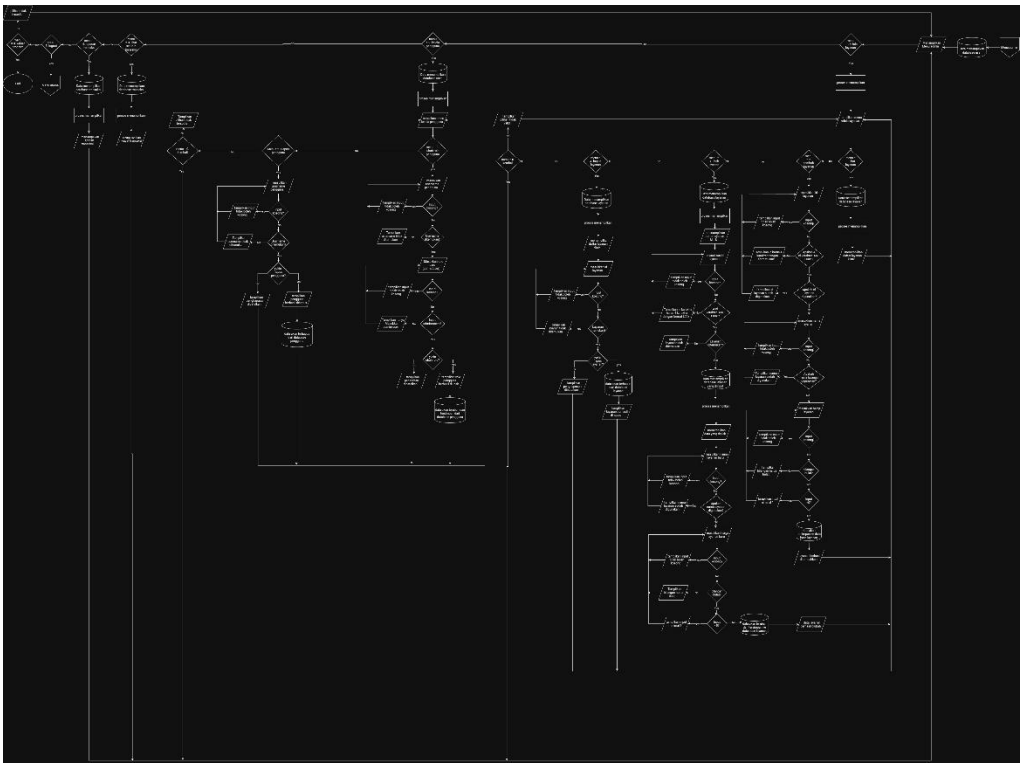
7. **Menu Logout**

   Jika pengguna memilih menu logout, sistem mengakhiri sesi akun yang sedang digunakan dan mengarahkan pengguna kembali ke menu utama.

8. **Menu Keluar Program**

   Jika pengguna memilih menu keluar program, sistem mengakhiri seluruh proses program. Tahap ini menandai berakhirnya penggunaan aplikasi.

2.4 Flowchart Menu Admin



Gambar 2. 3. Flowchart Menu Admin

1. **Menampilkan Data Admin**

   Setelah admin berhasil login, sistem mengambil data akun dari database users. Data tersebut kemudian diproses dan ditampilkan sebagai informasi akun admin. Selanjutnya, sistem menampilkan menu admin sebagai akses untuk mengelola data layanan, data pengguna, dan transaksi klinik.

2. **Menampilkan Menu Admin**

   Sistem menampilkan beberapa pilihan menu yang dapat digunakan oleh admin untuk mengelola data dan aktivitas pada sistem Administrasi Transaksi Klinik, yaitu:

   a. Kelola Layanan
   b. Kelola Pengguna
   c. Lihat Seluruh Transaksi
   d. Laporan Transaksi
   e. Logout
   f. Keluar Program

Admin diminta untuk memasukkan pilihan sesuai dengan menu yang tersedia. Jika pilihan tidak sesuai, sistem akan menampilkan bahwa pilihan tidak tersedia, kemudian mengarahkan admin kembali ke menu admin.

3. Menu Kelola Layanan

Jika admin memilih menu kelola layanan, sistem mengambil data layanan dari database, kemudian memproses dan menampilkan daftar layanan klinik. Pada menu ini, admin dapat memilih beberapa tindakan, yaitu melihat, menambahkan, mengubah, dan menghapus layanan.

a. Melihat Layanan

Sistem mengambil seluruh data layanan dari database, kemudian memproses dan menampilkan daftar layanan kepada admin. Setelah data ditampilkan, admin dapat kembali ke menu kelola layanan.

b. Menambah Layanan

Admin diminta memasukkan ID layanan baru. Sistem memeriksa apakah input kosong, memastikan format ID sesuai ketentuan, serta memeriksa apakah ID sudah digunakan. Jika tidak sesuai, sistem menampilkan pesan kesalahan dan meminta admin memasukkan kembali ID. Jika valid, admin diminta memasukkan nama layanan dan harga layanan. Sistem memeriksa kelengkapan nama serta memastikan harga berupa bilangan bulat dan memenuhi ketentuan. Jika seluruh input valid, data layanan disimpan ke database dan sistem menampilkan pesan bahwa layanan berhasil ditambahkan.

c. Mengubah Layanan

Admin diminta memasukkan ID layanan yang ingin diubah. Sistem memeriksa kelengkapan input dan memastikan ID layanan ditemukan dalam database. Jika tidak ditemukan, sistem menampilkan pesan kesalahan. Jika ditemukan, sistem menampilkan data layanan yang dipilih. Admin kemudian memasukkan nama dan harga layanan yang baru. Sistem memvalidasi setiap input sebelum menyimpan perubahan ke database. Jika seluruh data sesuai ketentuan, sistem menampilkan pesan bahwa layanan berhasil diperbarui.

d. Menghapus Layanan

Admin diminta memasukkan ID layanan yang ingin dihapus. Sistem memeriksa apakah input kosong dan memastikan layanan tersedia dalam database. Jika layanan ditemukan, sistem meminta konfirmasi penghapusan. Apabila admin menyetujui, data layanan dihapus dari database dan sistem menampilkan pesan keberhasilan. Jika penghapusan dibatalkan, data tetap tersimpan.

e. Kembali

Digunakan untuk keluar dari submenu Kelola Layanan dan kembali ke halaman Menu Admin. Jika admin memasukkan pilihan yang tidak tersedia, sistem menampilkan pesan bahwa pilihan tidak valid, kemudian meminta admin memilih kembali.

4. Menu Kelola Pengguna

Jika admin memilih menu kelola pengguna, sistem mengambil data pengguna dari database users, kemudian memproses dan menampilkannya. Pada menu ini, admin dapat melakukan perubahan role dan penghapusan akun pengguna.

a. Mengubah Role Pengguna

Admin diminta memasukkan username pengguna yang ingin diubah. Sistem memeriksa apakah input kosong dan memastikan username ditemukan dalam database. Jika tidak ditemukan, sistem menampilkan pesan kesalahan. Jika ditemukan, admin diminta menentukan role baru, yaitu admin atau user. Sistem memeriksa kesesuaian input sebelum memperbarui data role pengguna pada database users.

b. Menghapus Pengguna

Admin diminta memasukkan username pengguna yang ingin dihapus. Sistem memeriksa kelengkapan input dan memastikan username tersedia dalam database. Jika ditemukan, sistem meminta konfirmasi penghapusan. Apabila admin menyetujui, data pengguna dihapus dari database users. Jika tidak disetujui, proses penghapusan dibatalkan dan data tetap tersimpan.

c. Kembali

Digunakan untuk keluar dari submenu Kelola Pengguna dan kembali ke halaman Menu Admin. Jika admin memasukkan pilihan yang tidak tersedia, sistem menampilkan pesan bahwa pilihan tidak valid dan meminta admin memasukkan pilihan kembali.

5. Menu Lihat Seluruh Transaksi

Jika admin memilih menu lihat seluruh transaksi, sistem mengambil seluruh data dari database transaksi, kemudian memproses dan menampilkan riwayat transaksi yang tersimpan. Informasi tersebut dapat digunakan admin untuk memeriksa aktivitas transaksi yang telah dilakukan pengguna. Setelah data ditampilkan, admin dapat kembali ke menu admin.

6. Menu Laporan Transaksi

Jika admin memilih menu laporan transaksi, sistem mengambil data transaksi dari database untuk diproses menjadi laporan. Data transaksi tersebut diolah dan ditampilkan sebagai informasi rekapitulasi aktivitas transaksi klinik. Laporan ini membantu admin memantau jumlah transaksi serta informasi pembayaran yang tercatat dalam sistem. Setelah laporan ditampilkan, admin dapat kembali ke menu admin.

7. Logout

Jika admin memilih menu logout, sistem mengakhiri sesi akun admin yang sedang digunakan dan mengarahkan pengguna kembali ke menu utama. Untuk mengakses menu admin kembali, pengguna harus melakukan login ulang.

8. Keluar Program

Jika admin memilih menu keluar program, sistem mengakhiri seluruh proses program. Tahap ini menandai berakhirnya penggunaan sistem.

BAB III
HASIL DAN PEMBAHASAN

3.1 Implementasi Program

Program Administrasi Transaksi Klinik dapat diimplementasikan dalam kehidupan nyata dengan digunakan oleh klinik untuk membantu proses administrasi layanan dan transaksi secara digital. Dalam penerapannya, admin klinik bertugas mengelola daftar layanan beserta harganya, mengatur akun pengguna, memantau seluruh transaksi, dan melihat laporan transaksi. Sementara itu, pasien atau pengguna dapat membuat akun, login ke sistem, melihat daftar layanan beserta biayanya, melakukan top-up saldo, membayar layanan menggunakan saldo yang tersedia, serta melihat riwayat transaksi dan bukti pembayaran berupa invoice.

Program ini dapat dijalankan pada komputer atau laptop di bagian administrasi klinik, dengan data pengguna, layanan, dan transaksi disimpan dalam file CSV sehingga dapat dibuka kembali saat program dijalankan. Untuk diterapkan secara langsung di klinik, program ini perlu dikembangkan lebih lanjut dengan menggunakan basis data yang lebih aman, sistem jaringan agar dapat diakses oleh beberapa perangkat, serta perlindungan data pribadi pasien. Selain itu, fitur pembayaran saldo perlu diintegrasikan dengan sistem pembayaran yang sah apabila klinik ingin menerima pembayaran digital secara nyata. Dengan demikian, program ini dapat menjadi dasar pengembangan sistem administrasi klinik yang membantu mempermudah pengelolaan layanan, mempercepat pencatatan transaksi, dan mengurangi kesalahan pencatatan secara manual.

3.2 Alur Program

3.2.1. Import library

```
1  import csv
2  import os
3  import time
4  import random
5  import hashlib
6  from datetime import datetime
7  from prettytable import PrettyTable
8  from pwinput import pwinput
```

Gambar 3. 1. Import Library

Penjelasan setiap library:

1. csv: digunakan untuk membaca dan menyimpan data dalam format CSV. Program memanfaatkannya untuk menyimpan data pengguna, layanan klinik, dan transaksi.
2. os: digunakan untuk mengelola sistem operasi, seperti membersihkan layar terminal, memeriksa keberadaan file, serta menentukan lokasi penyimpanan File.
3. time: digunakan untuk memberikan jeda waktu dalam program menggunakan fungsi time.sleep().
4. random: digunakan untuk menghasilkan angka atau pilihan secara acak.
5. hashlib: digunakan untuk mengubah password menjadi bentuk hash menggunakan algoritma SHA-256 sehingga password tidak disimpan dalam bentuk teks asli.
6. datetime: digunakan untuk mengambil tanggal dan waktu saat transaksi dilakukan.
7. PrettyTable: digunakan untuk menampilkan data dalam bentuk tabel agar lebih rapi dan mudah dibaca.
8. pwinput: digunakan untuk menerima input password tanpa memperlihatkan karakter password yang diketik pengguna.

```
9    FOLDER_PROGRAM = os.path.dirname(os.path.abspath(__file__))
```

Gambar 3. 2. Tempat Folder

Kode tersebut berfungsi untuk mendapatkan lokasi folder tempat file Python program disimpan. Hasilnya disimpan dalam variabel FOLDER_PROGRAM.

3.2.2. Pengaturan lokasi File csv

```
# Pengaturan lokasi file CSV
def lokasi_csv(nama_file):
    lokasi_program = os.path.join(FOLDER_PROGRAM, nama_file)
    lokasi_terminal = os.path.abspath(nama_file)
    if os.path.isfile(lokasi_program):
        return lokasi_program
    if os.path.isfile(lokasi_terminal):
        return lokasi_terminal
    return lokasi_program
```

Gambar 3. 3. Lokasi File CSV

Setelah melakukan import library, program menentukan lokasi penyimpanan file menggunakan variabel FOLDER_PROGRAM yang diperoleh melalui fungsi os.path.dirname(os.path.abspath(__file__)). Kode tersebut berfungsi untuk mengetahui lokasi folder tempat file Python program disimpan. Selanjutnya, terdapat fungsi lokasi_csv(nama_file) yang digunakan untuk menentukan lokasi file CSV yang akan digunakan.

Fungsi ini memeriksa apakah file tersedia di dalam folder program atau di lokasi kerja terminal. Pemeriksaan dilakukan menggunakan os.path.isfile(), sedangkan os.path.join() dan os.path.abspath() digunakan untuk membentuk alamat file secara lengkap. Apabila file ditemukan, program akan menggunakan lokasi tersebut. Namun, jika file belum tersedia, program akan menentukan lokasi folder program sebagai tempat penyimpanan file baru.

```
FILE_USERS = lokasi_csv("users.csv")
FILE_LAYANAN = lokasi_csv("layanan.csv")
FILE_TRANSAKSI = lokasi_csv("transaksi.csv")
ROLE_PENGGUNA = ("admin", "user")
```

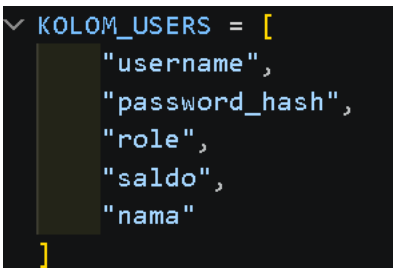
Gambar 3. 4. Lokasi File dan Role

Hasil dari fungsi tersebut kemudian digunakan untuk menentukan lokasi tiga file utama, yaitu users.csv, layanan.csv, dan transaksi.csv. File users.csv digunakan untuk menyimpan informasi akun pengguna, File layanan.csv digunakan untuk menyimpan daftar layanan klinik beserta harganya, sedangkan file transaksi.csv digunakan untuk mencatat seluruh aktivitas transaksi yang dilakukan pengguna. Selain itu, terdapat variabel ROLE_PENGGUNA yang berisi dua jenis hak akses, yaitu admin dan user. Variabel ini digunakan untuk membatasi jenis akun yang dapat mengakses sistem.

3.2.3. Menentukan Struktur Kolom Data

Program menggunakan tiga file CSV yang masing masing memiliki struktur kolom berbeda.

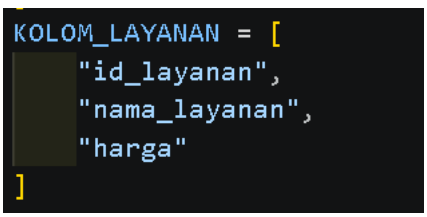
a. Kolom users



Gambar 3. 5. Kolom Users

Variabel KOLOM_USERS digunakan untuk menentukan informasi yang akan disimpan dalam data pengguna, seperti username, password hash, role, saldo, dan nama lengkap. Dengan adanya struktur tersebut, setiap akun pengguna dapat memiliki identitas dan informasi saldo yang tersimpan secara teratur.

b. Kolom layanan



Gambar 3. 6. Kolom Layanan

Variabel KOLOM_LAYANAN digunakan untuk menentukan informasi yang terdapat dalam file layanan, yaitu ID layanan, nama layanan, dan harga. Struktur ini memungkinkan admin untuk mengelola berbagai jenis layanan kesehatan yang tersedia di klinik.

c. Kolom transaksi

```
KOLOM_TRANSAKSI = [
    "id_transaksi",
    "waktu",
    "username",
    "jenis",
    "id_layanan",
    "nama_layanan",
    "jumlah",
    "nominal",
    "saldo_awal",
    "saldo_akhir",
    "status"
]
```

Gambar 3. 7. Kolom Transaksi

Variabel KOLOM_TRANSAKSI digunakan untuk menentukan informasi yang perlu dicatat ketika transaksi dilakukan. Data tersebut mencakup ID transaksi, waktu transaksi, username, jenis transaksi, ID layanan, nama layanan, jumlah layanan, nominal transaksi, saldo awal, saldo akhir, dan status transaksi.

Dengan adanya pembagian struktur kolom tersebut, program dapat menyimpan setiap informasi sesuai dengan kategori datanya sehingga proses pembacaan dan pengelolaan data menjadi lebih teratur.

3.2.4. Fungsi pendukung program

Terdapat beberapa Fungsi pendukung yang digunakan untuk mempermudah jalannya program. Fungsi membersihkan layar:

```
def bersihkan_layar():
    if os.name == "nt":
        os.system("cls")
    else:
        os.system("clear")
```

Gambar 3. 8. Fungsi Membersihkan Layar

Fungsi bersihkan_layar() digunakan untuk membersihkan tampilan terminal sebelum program menampilkan menu baru. Fungsi ini menggunakan percabangan untuk memeriksa sistem operasi yang digunakan. Apabila program dijalankan pada Windows, sistem akan menggunakan perintah cls, sedangkan pada Linux atau macOS menggunakan perintah clear. Dengan demikian, tampilan program menjadi lebih rapi karena informasi dari menu sebelumnya tidak terus memenuhi layar.

a. Fungsi jeda

```
def jeda():
    input("\nTekan enter untuk melanjutkan...")
```

Gambar 3. 9. Fungsi Jeda

Fungsi jeda() yang digunakan untuk menghentikan sementara jalannya program sampai pengguna menekan tombol Enter. Fungsi ini bertujuan memberikan waktu kepada pengguna untuk membaca informasi yang ditampilkan sebelum program melanjutkan proses berikutnya.

b. Fungsi input teks dan validasi

```
def input_teks(pesan):
    while True:
        try:
            nilai = input(pesan).strip()
            if nilai:
                return nilai
            print("Input tidak boleh kosong. Silakan isi terlebih dahulu ⚠️")
        except (ValueError, TypeError):
            print("⚠️ Input tidak valid. Silakan coba lagi. ⚠️")
```

Gambar 3. 10. Fungsi input teks dan validasi

Fungsi input_teks(pesan) digunakan untuk menerima masukan berupa teks dari pengguna. Di dalamnya terdapat perulangan while True agar proses input terus dilakukan sampai pengguna memasukkan teks

yang valid. Metode .strip() digunakan untuk menghilangkan spasi di awal dan akhir masukan, sedangkan kondisi if digunakan untuk memastikan bahwa input tidak kosong.

c. Fungsi input password

```
def input_password(pesan):
    while True:
        try:
            password = pwinput(pesan)
            if password.strip():
                return password
            print("⚠ Password tidak boleh kosong. ⚠")
        except (ValueError, TypeError):
            print("⚠ Input password tidak valid. Silakan coba lagi. ⚠")
```

Gambar 3. 11. Fungsi input password

Program juga memiliki fungsi input_password(pesan) yang digunakan untuk menerima password melalui library pwinput. Berbeda dengan input teks biasa, fungsi ini menyembunyikan karakter password yang dimasukkan pengguna.

d. Fungsi input angka

```
def input_angka(pesan, minimum=None):
    while True:
        try:
            angka = int(input_teks(pesan))
            if minimum is not None and angka < minimum:
                print("⚠ Input minimal adalah", minimum, "⚠")
            else:
                return angka
        except ValueError:
            print("Input harus berupa angka bulat ⚠")
```

Gambar 3. 12. Fungsi input angka

Fungsi input_angka(pesan, minimum=None) yang digunakan untuk menerima masukan berupa angka bulat. Fungsi ini memanFaatkan int() untuk mengubah input menjadi bilangan bulat dan menggunakan try-except untuk menangani kesalahan apabila pengguna memasukkan karakter yang bukan angka. Parameter minimum digunakan untuk menentukan batas minimal angka yang diperbolehkan.

e. Fungsi konfirmasi

```
def konfirmasi(pesan):
    while True:
        pilihan = input_teks(pesan + " (y/n): ").lower()
        if pilihan == "y":
            return True
        if pilihan == "n":
            return False
        print("Masukkan y atau n ⚠️")
```

Gambar 3. 13. Fungsi konfirmasi

Fungsi konfirmasi(pesan) digunakan untuk meminta persetujuan pengguna sebelum suatu tindakan dijalankan. Pengguna dapat memasukkan huruf y untuk menyetujui atau n untuk membatalkan tindakan. Jika input tidak sesuai, program akan meminta pengguna memasukkan pilihan kembali.

f. Fungsi format rupiah

```
def format_rupiah(nominal):
    return "Rp" + format(int(nominal), ",").replace(",", ".")
```

Gambar 3. 14. Fungsi format rupiah

Fungsi format_rupiah(nominal) yang digunakan untuk mengubah angka menjadi format mata uang rupiah. Contohnya, angka 50000 akan ditampilkan menjadi Rp50.000 sehingga informasi nominal transaksi lebih mudah dipahami.

3.2.5. Fungsi Tampilan Judul dan Tabel

Fungsi ini digunakan untuk memperindah tampilan program agar informasi yang ditampilkan lebih terstruktur.

a. Fungsi menampilkan judul

```
# Tampilan judul dan tabel
def tampilkan_judul(teks, ikon="✨", lebar=44):
    isi = f"{ikon} {teks} {ikon}"
    ruang = lebar - 4
    if len(isi) > ruang:
        ruang = len(isi)
        lebar = ruang + 4
    kiri = (ruang - len(isi)) // 2
    kanan = ruang - len(isi) - kiri
    print("┌" + "─" * (lebar - 2) + "┐")
    print("│ " + " " * kiri + isi + " " * kanan + "")
    print("└" + "─" * (lebar - 2) + "┘")
```

Gambar 3. 15. Fungsi menampilkan judul

Program memiliki fungsi tampilkan_judul(teks, ikon="✨", lebar=44) yang digunakan untuk membuat tampilan judul pada setiap menu. Fungsi ini menerima parameter berupa teks judul, ikon, dan ukuran lebar tampilan. Variabel isi digunakan untuk menggabungkan ikon dengan teks yang akan ditampilkan, sedangkan fungsi len() digunakan untuk menghitung panjang karakter judul.

Apabila panjang judul melebihi ukuran yang ditentukan, program akan menyesuaikan lebar bingkai secara otomatis. Selanjutnya, perhitungan pada variabel kiri dan kanan digunakan untuk mengatur jarak kosong agar teks berada di tengah bingkai. Hasilnya kemudian ditampilkan menggunakan karakter garis sehingga tampilan setiap menu terlihat lebih menarik dan terstruktur.

b. Fungsi menampilkan table



Gambar 3. 16. Fungsi menampilkan table

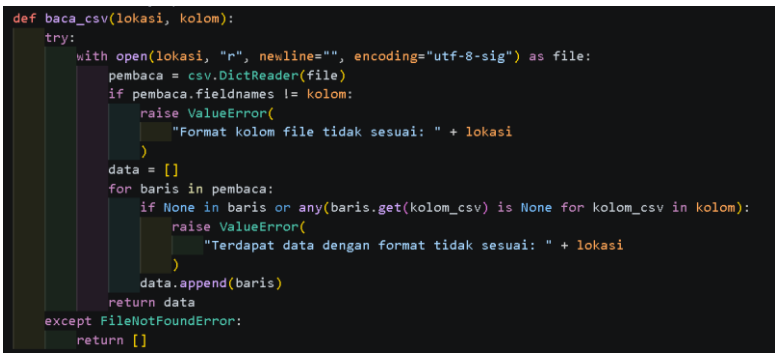
Fungsi tampilkan_tabel(judul, kolom, data) digunakan untuk menyajikan informasi dalam bentuk tabel menggunakan library PrettyTable. Fungsi ini membuat objek tabel baru, menentukan judul dan nama kolom, serta mengatur posisi teks agar rata kiri. Program juga melakukan pemeriksaan terhadap nama kolom yang mengandung kata harga, nominal, saldo, atau total.

Apabila ditemukan, posisi isi kolom tersebut akan diubah menjadi rata kanan agar angka terlihat lebih teratur. Selanjutnya, perulangan for digunakan untuk memasukkan setiap baris data ke dalam tabel melalui fungsi add_row(). Setelah seluruh data dimasukkan, tabel akan ditampilkan menggunakan print(). Fungsi ini digunakan pada berbagai bagian program, seperti menampilkan daftar layanan, rincian tagihan, daftar pengguna, dan laporan transaksi.

3.2.6. Fungsi membaca dan menyimpan file CSV

Fungsi ini merupakan salah satu bagian penting dalam program karena berhubungan langsung dengan penyimpanan data.

a. Fungsi membaca File CSV



```
def baca_csv(lokasi, kolom):
    try:
        with open(lokasi, "r", newline="", encoding="utf-8-sig") as file:
            pembaca = csv.DictReader(file)
            if pembaca.fieldnames != kolom:
                raise ValueError(
                    "Format kolom file tidak sesuai: " + lokasi
                )
            data = []
            for baris in pembaca:
                if None in baris or any(baris.get(kolom_csv) is None for kolom_csv in kolom):
                    raise ValueError(
                        "Terdapat data dengan format tidak sesuai: " + lokasi
                    )
                data.append(baris)
            return data
    except FileNotFoundError:
        return []
```

Gambar 3. 17. Fungsi membaca file CSV

Fungsi ini menerima parameter berupa lokasi file dan daftar nama kolom yang harus tersedia. Proses pembacaan dilakukan menggunakan open() dengan mode "r" yang berarti file dibuka untuk dibaca. Selanjutnya, csv.DictReader() digunakan untuk membaca setiap baris file dan mengubahnya menjadi bentuk dictionary sehingga data dapat diakses berdasarkan nama kolomnya.

Setelah file dibaca, program memeriksa apakah nama kolom yang tersedia sudah sesuai dengan struktur yang telah ditentukan. Pemeriksaan dilakukan menggunakan pembaca.fieldnames. Jika susunan kolom tidak sesuai, program akan menghasilkan kesalahan ValueError. Selanjutnya, perulangan digunakan untuk memeriksa setiap baris data dan memastikan tidak terdapat kolom atau nilai yang tidak sesuai. Setiap baris yang valid kemudian dimasukkan ke dalam variabel data menggunakan append(). Setelah seluruh data berhasil dibaca, fungsi mengembalikan daftar data tersebut menggunakan return data. Apabila file belum ditemukan, program akan menangani kondisi tersebut melalui except FileNotFoundError dan mengembalikan daftar kosong.

b. Fungsi menyimpan File CSV

```
def simpan_csv(lokasi, kolom, data):
    with open(lokasi, "w", newline="", encoding="utf-8") as file:
        penulis = csv.DictWriter(file, fieldnames=kolom)
        penulis.writeheader()
        penulis.writerows(data)
```

Gambar 3. 18. Fungsi menyimpan file CSV

Fungsi simpan_csv(lokasi, kolom, data) digunakan untuk menyimpan data ke dalam file CSV. Fungsi ini membuka file menggunakan mode "w" yang berarti menulis atau memperbarui isi file. Kemudian, csv.DictWriter() digunakan untuk menuliskan data dalam bentuk dictionary berdasarkan struktur kolom yang sudah ditentukan. Fungsi writeheader() digunakan untuk menuliskan nama kolom pada bagian pertama file, sedangkan writerows() digunakan untuk menyimpan seluruh baris data. Dengan adanya kedua fungsi tersebut, program dapat membaca data yang sudah tersimpan sekaligus memperbarui informasi ketika terdapat penambahan, perubahan, atau penghapusan data.

c. Fungsi membaca data layanan

```
def baca_csv_layanan():
    try:
        with open(FILE_LAYANAN, "r", newline="", encoding="utf-8-sig") as file:
            pembaca = csv.DictReader(file)
            kolom_file = pembaca.fieldnames or []
            if not all(kolom in kolom_file for kolom in ("id_layanan", "nama_layanan", "harga")):
                raise ValueError("Format kolom file layanan.csv tidak sesuai.")
            data_layanan = []
            for baris in pembaca:
                data_layanan.append({
                    "id_layanan": (baris.get("id_layanan") or "").strip(),
                    "nama_layanan": (baris.get("nama_layanan") or "").strip(),
                    "harga": (baris.get("harga") or "").strip()
                })
        return data_layanan
    except FileNotFoundError:
        return []
```

Gambar 3. 19. Fungsi membaca data layanan

Fungsi baca_csv_layanan() yang secara khusus digunakan untuk membaca data layanan dari file layanan.csv. Fungsi ini memeriksa apakah kolom ID layanan, nama layanan, dan harga sudah tersedia. Data yang dibaca kemudian dimasukkan ke dalam dictionary baru menggunakan get() dan strip() agar nilai yang disimpan sesuai dengan format yang diharapkan.

d. Fungsi menyimpan data layanan

```
def simpan_csv_layanan(data_layanan):
    simpan_csv(FILE_LAYANAN, KOLOM_LAYANAN, data_layanan)
```

Gambar 3. 20. Fungsi menyimpan data layanan

Fungsi simpan_csv_layanan() digunakan untuk menyimpan data layanan dengan memanfaatkan fungsi simpan_csv() yang telah dibuat sebelumnya.

e. Fungsi kunci pengurutan ID layanan

```
def kunci_id_layanan(layanan):
    return layanan["id_layanan"].upper()
```

Gambar 3. 21. Fungsi kunci pengurutan ID layanan

Fungsi kunci_id_layanan() yang digunakan untuk mengubah ID layanan menjadi huruf kapital sebagai dasar pengurutan data layanan agar susunannya lebih teratur.

3.2.7. Pengolahan data akun dan data awal

a **Fungsi hash password**

```
def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()
```

Gambar 3. 22. Fungsi hash password

Fungsi hash password digunakan agar password yang tersimpan dalam bentuk kode acak

b **Fungsi insialisasi data**

```
def inisialisasi_data():
    if not os.path.exists(FILE_USERS):
        data_users = [
            {
                "username": "admin",
                "password_hash": hash_password("1"),
                "role": "admin",
                "saldo": "0",
                "nama": "Administrator Klinik"
            }
        ]
        simpan_csv(FILE_USERS, KOLOM_USERS, data_users)
```

Gambar 3. 23. Fungsi insialisasi data

Fungsi inisialisasi_data() yang digunakan untuk mempersiapkan data awal sebelum program dijalankan. Fungsi ini memeriksa apakah file users.csv sudah tersedia menggunakan os.path.exists(). Apabila file belum ditemukan, program akan membuat data akun admin utama yang berisi username, password yang sudah di-hash, role admin, saldo awal nol, dan nama administrator klinik. Data tersebut kemudian disimpan menggunakan fungsi simpan_csv().

```
else:
    data_users = baca_csv(FILE_USERS, KOLOM_USERS)
    if cari_user("admin", data_users) is None:
        data_users.append({
            "username": "admin",
            "password_hash": hash_password("Adminklinik2"),
            "role": "admin",
            "saldo": "0",
            "nama": "Administrator Klinik"
        })
        simpan_csv(FILE_USERS, KOLOM_USERS, data_users)
```

Gambar 3. 24. Membaca data user

Apabila file pengguna sudah tersedia, program akan membaca data akun yang tersimpan dan memeriksa apakah akun admin utama sudah terdaftar. Jika akun admin belum ditemukan, program akan menambahkan akun tersebut ke dalam daftar pengguna dan menyimpannya kembali.

```
data_layanan_awal = [
    {"id_layanan": "L001", "nama_layanan": "Konsultasi Umum", "harga": "50000"},
    {"id_layanan": "L002", "nama_layanan": "Pemeriksaan Tekanan Darah", "harga": "20000"},
    {"id_layanan": "L003", "nama_layanan": "Pemeriksaan Gula Darah", "harga": "35000"},
    {"id_layanan": "L004", "nama_layanan": "Pemeriksaan Kolesterol", "harga": "45000"},
    {"id_layanan": "L005", "nama_layanan": "Konsultasi Kesehatan", "harga": "40000"}
]
```

Gambar 3. 25. Data daftar layanan

Setelah mempersiapkan data pengguna, fungsi ini juga membuat daftar layanan awal yang berisi beberapa jenis layanan kesehatan, seperti konsultasi umum, pemeriksaan tekanan darah, pemeriksaan gula darah, pemeriksaan kolesterol, dan konsultasi kesehatan beserta harga masing-masing layanan.

```
perlu_buat_layanan = not os.path.exists(FILE_LAYANAN)
if not perlu_buat_layanan:
    try:
        data_layanan_tersimpan = baca_csv_layanan()
        if not data_layanan_tersimpan:
            perlu_buat_layanan = True
        else:
            data_layanan_tersimpan.sort(key=kunci_id_layanan)
            simpan_csv_layanan(data_layanan_tersimpan)
    except (ValueError, csv.Error, OSError):
        perlu_buat_layanan = True
if perlu_buat_layanan:
    simpan_csv_layanan(data_layanan_awal)
```

Gambar 3. 26. Memeriksa file layanna yang tersedia

Program kemudian memeriksa keberadaan file layanan.csv. Jika file belum tersedia atau belum memiliki data, daftar layanan awal akan disimpan ke dalam file tersebut. Namun, apabila file sudah memiliki data, program akan membaca dan mengurutkan layanan berdasarkan ID menggunakan fungsi kunci_id_layanan(). Dengan demikian, fungsi inisialisasi_data() bertujuan memastikan bahwa file dan data awal yang dibutuhkan sudah tersedia sebelum pengguna mulai menggunakan sistem.

3.2.8. Fungsi Pencarian Data Pengguna dan Layanan

Program memiliki dua fungsi pencarian utama, yaitu cari_user(username, data_users) dan cari_layanan(id_layanan, data_layanan).

a. Mencari pengguna

```
def cari_user(username, data_users):
    for user in data_users:
        if user["username"].lower() == username.lower():
            return user
    return None
```

Gambar 3. 27. Fungsi mencari pengguna

Fungsi cari_user() digunakan untuk mencari akun pengguna berdasarkan username yang dimasukkan. Proses pencarian dilakukan menggunakan perulangan for untuk memeriksa setiap data akun yang terdapat dalam daftar pengguna. Kemudian, program membandingkan username yang dimasukkan dengan username yang tersimpan

menggunakan metode .lower() agar proses pencarian tidak membedakan huruf besar dan kecil. Jika ditemukan username yang sesuai, fungsi akan mengembalikan seluruh informasi akun tersebut. Namun, jika tidak ditemukan, fungsi akan mengembalikan nilai **None.**

b. Mencari layanan

```
def cari_layanan(id_layanan, data_layanan):
    for layanan in data_layanan:
        if layanan["id_layanan"].lower() == id_layanan.lower():
            return layanan
    return None
```

Gambar 3. 28. Fungsi mencari layanan

Fungsi cari_layanan() memiliki cara kerja yang hampir sama, tetapi digunakan untuk mencari data layanan berdasarkan ID layanan. Program akan memeriksa setiap data layanan dan membandingkan ID yang dimasukkan dengan ID yang tersimpan. Apabila ditemukan kecocokan, informasi layanan akan dikembalikan sehingga dapat digunakan dalam proses selanjutnya. Kedua fungsi ini sangat membantu karena dapat digunakan berulang kali pada berbagai bagian program tanpa harus menuliskan proses pencarian yang sama.

3.2.9. Registrasi Pengguna

```
def registrasi():
    bersihkan_layar()
    tampilkan_judul("Registrasi Pengguna", "📝")
    data_users = baca_csv(FILE_USERS, KOLOM_USERS)
```

Gambar 3. 29. Fungsi registrasi pengguna

Fungsi registrasi() digunakan untuk memungkinkan pengguna membuat akun baru agar dapat mengakses sistem Administrasi Transaksi Klinik. Pada awal fungsi, program membersihkan layar, menampilkan judul registrasi, dan membaca seluruh data pengguna yang sudah tersimpan di dalam file CSV.

a. Membuat username dan password

```
while True:
    username = input_teks("Buat username: ")
    if len(username) < 4:
        print("Username minimal 4 karakter!")
    elif not username.isalnum():
        print("Username hanya boleh menggunakan huruf dan angka!")
    elif cari_user(username, data_users) is not None:
        print("Username sudah digunakan!")
    else:
        break
while True:
    password = input_password("Buat password (minimal 8 karakter): ")
    if len(password) < 8:
        print("Password minimal 8 karakter.")
    else:
        password_konfirmasi = input_password("Konfirmasi password: ")
        if password != password_konfirmasi:
            print("Konfirmasi password tidak sesuai.")
        else:
            break
```

Gambar 3. 30. Membuat username dan password

Pengguna diminta membuat username melalui fungsi input_teks(). Username yang dimasukkan harus memenuhi beberapa ketentuan, yaitu memiliki minimal empat karakter, hanya menggunakan huruf dan angka, serta belum pernah digunakan oleh pengguna lain. Pemeriksaan dilakukan menggunakan len(), isalnum(), dan fungsi cari_user().

Jika username tidak memenuhi ketentuan, program akan meminta pengguna memasukkan username kembali. Setelah username berhasil dibuat, program meminta pengguna memasukkan password melalui fungsi input_password(). Password harus memiliki minimal delapan karakter dan harus dimasukkan kembali untuk memastikan bahwa password konfirmasi sesuai dengan password pertama. Jika kedua password berbeda, program akan meminta pengguna mengulang proses tersebut.

b. Masukkan nama lengkap dan buat akun baru

```
while True:
    nama = input_teks("Nama lengkap: ")
    if nama == "":
        print("Nama lengkap tidak boleh kosong!")
    else:
        break
user_baru = {
    "username": username,
    "password_hash": hash_password(password),
    "role": "user",
    "saldo": "0",
    "nama": nama
}
data_users.append(user_baru)
try:
    simpan_csv(FILE_USERS, KOLOM_USERS, data_users)
    print("\n✅ Registrasi berhasil! ✅")
    print("Silakan login menggunakan akun yang telah dibuat.")
    jeda()
except OSError:
    data_users.remove(user_baru)
    print("Registrasi gagal karena data tidak dapat disimpan.")
    jeda()
```

Gambar 3. 31. Masukkan nama lengkap dan buat akun baru

Selanjutnya, pengguna diminta memasukkan nama lengkap yang nantinya digunakan sebagai identitas dalam sistem. Setelah seluruh informasi berhasil dimasukkan, program menyusun data akun baru dalam bentuk dictionary yang berisi username, password hash, role, saldo awal, dan nama lengkap. Role secara otomatis ditetapkan sebagai user, sedangkan saldo awal diberikan sebesar nol.

Data akun baru kemudian dimasukkan ke dalam daftar pengguna menggunakan append() dan disimpan ke dalam file users.csv melalui fungsi simpan_csv(). Jika proses penyimpanan berhasil, program menampilkan pesan bahwa registrasi telah berhasil dan pengguna dapat melanjutkan ke proses login. Namun, jika terjadi kesalahan ketika menyimpan data, program menggunakan remove() untuk mengeluarkan akun baru dari daftar sementara sehingga data tidak dianggap berhasil ditambahkan apabila penyimpanan gagal.

3.2.10. Login Pengguna

```
def login():
    data_users = baca_csv(FILE_USERS, KOLOM_USERS)
    tampilkan_judul("Login", "🔐")
    percobaan = 0
    while percobaan < 3:
        username = input_teks("Username: ")
        password = input_password("Password: ")
        user = cari_user(username, data_users)
        if user is not None:
            if user["password_hash"] == hash_password(password):
                if user["role"] not in ROLE_PENGGUNA:
                    print("Role akun tidak valid.")
                    jeda()
                    return None
                try:
                    int(user["saldo"])
                except (ValueError, TypeError):
                    print("Saldo akun tidak valid. Periksa data pengguna.")
                    jeda()
                    return None
                print("\n✅ Login berhasil. ✅")
                print("\n👋 Selamat datang,", user["nama"], "👋")
                time.sleep(1)
                return user
```

Gambar 3. 32. Login pengguna

```
    percobaan += 1
    sisa = 3 - percobaan
    print("❌ Username atau password salah. ❌")
    if sisa > 0:
        print("Sisa percobaan:", sisa)
print("\nBatas percobaan login telah habis!")
jeda()
return None
```

Gambar 3. 33. Percobaan login

Fungsi login() digunakan untuk memeriksa identitas pengguna sebelum memberikan akses ke dalam sistem. Pada awal proses, program membaca data pengguna dari File CSV, menampilkan judul login, dan membuat variabel percobaan dengan nilai awal nol. Selanjutnya, program meminta pengguna memasukkan username dan password. Username yang dimasukkan akan diperiksa menggunakan fungsi cari_user() untuk memastikan bahwa akun tersebut sudah terdaftar. Apabila akun ditemukan, program akan mengubah password yang dimasukkan menjadi bentuk hash menggunakan fungsi hash_password(), kemudian membandingkannya dengan password hash yang tersimpan di file CSV. Jika kedua nilai tersebut sesuai, program akan memeriksa apakah role

akun termasuk dalam daftar role yang diperbolehkan, yaitu admin atau user.

Program juga memeriksa apakah saldo akun dapat diubah menjadi angka bulat menggunakan int(). Pemeriksaan ini diperlukan karena saldo akan digunakan dalam proses perhitungan transaksi dan pembayaran layanan. Jika seluruh pemeriksaan berhasil, program menampilkan pesan login berhasil beserta nama pengguna dan memberikan jeda waktu menggunakan time.sleep(1). Setelah itu, fungsi mengembalikan data akun menggunakan return user. Namun, apabila username atau password tidak sesuai, variabel percobaan akan bertambah satu dan program menampilkan pesan kesalahan beserta jumlah kesempatan login yang masih tersedia. Proses ini akan terus dilakukan sampai pengguna berhasil login atau telah mencapai batas tiga kali percobaan. Jika seluruh kesempatan telah digunakan, program akan menampilkan pemberitahuan bahwa batas percobaan login telah habis.

3.2.11. Pengelolaan daftar layanan klinik

a. **Menampilkan Daftar Layanan Klinik**

```
def tampilkan_layanan():
    data_layanan = baca_csv_layanan()
    data_layanan.sort(key=kunci_id_layanan)
    isi_tabel = []
    for layanan in data_layanan:
        isi_tabel.append([
            layanan["id_layanan"],
            layanan["nama_layanan"],
            format_rupiah(layanan["harga"])
        ])
    if not isi_tabel:
        print("Belum ada layanan yang tersedia ⚠️")
        print("Silakan tambahkan layanan terlebih dahulu.")
    else:
        tampilkan_tabel(
            "✦  DAFTAR LAYANAN KLINIK  ✦",
            ["ID Layanan", "Nama Layanan", "Harga (Rp)"],
            isi_tabel
        )
```

Gambar 3. 34. Menampilkan daftar layanan

```
if not isi_tabel:
    print("Belum ada layanan yang tersedia ⚠️")
    print("Silakan tambahkan layanan terlebih dahulu.")
else:
    tampilkan_tabel(
        "✦  DAFTAR LAYANAN KLINIK  ✦",
        ["ID Layanan", "Nama Layanan", "Harga (Rp)"],
        isi_tabel
    )
```

Gambar 3. 35. Menambilkan tabel layanan

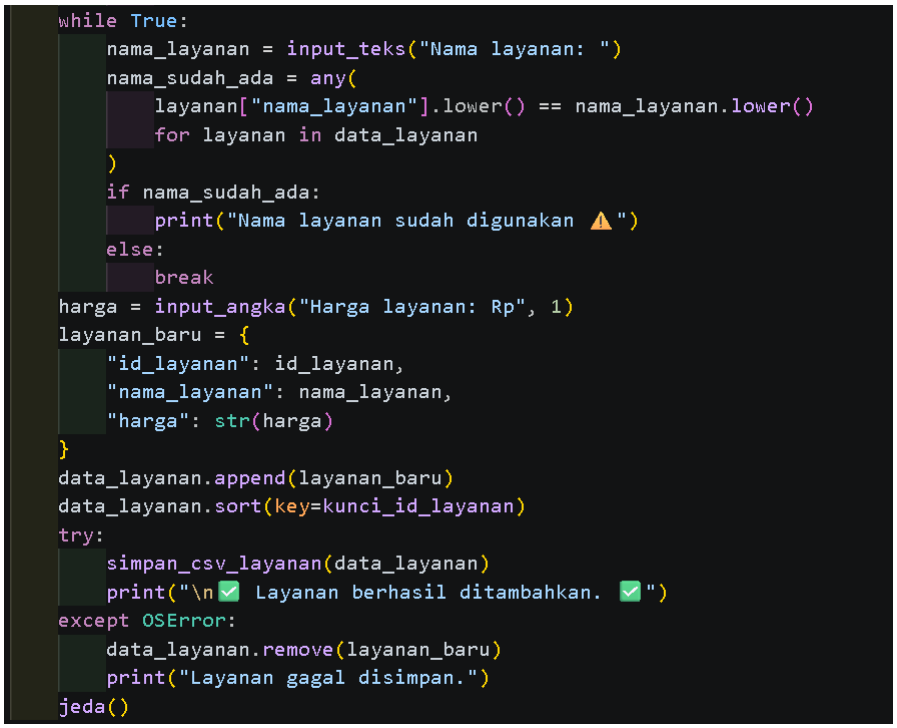
Program menyediakan beberapa fungsi untuk mengelola daftar layanan kesehatan yang tersedia di klinik. Fungsi pertama adalah tampilkan_layanan() yang digunakan untuk menampilkan seluruh.

Program menampilkan daftar layanan dalam bentuk tabel dengan menggunakan code seperti pada gambar 3.35 dan akan menampilkan belum ada layanan jika ternyata pada database belum memiliki layanan.

b. **Menambahkan Layanan**

```
def tambah_layanan():
    bersihkan_layar()
    tampilkan_judul("Tambah Layanan", "➕")
    data_layanan = baca_csv_layanan()
    while True:
        id_layanan = input_teks("ID layanan baru (L001): ").upper()
        if len(id_layanan) != 4 or not id_layanan.startswith("L00") or not id_layanan[3].isdigit():
            print("ID harus 4 karakter dengan format L00X ⚠️")
        elif cari_layanan(id_layanan, data_layanan) is not None:
            print("ID layanan sudah digunakan ⚠️")
        else:
            break
    while True:
        nama_layanan = input_teks("Nama layanan: ")
        nama_sudah_ada = any(
            layanan["nama_layanan"].lower() == nama_layanan.lower()
            for layanan in data_layanan
        )
        if nama_sudah_ada:
            print("Nama layanan sudah digunakan ⚠️")
        else:
            break
    harga = input_angka("Harga layanan: Rp", 1)
    layanan_baru = {
        "id_layanan": id_layanan,
        "nama_layanan": nama_layanan,
        "harga": str(harga)
```

Gambar 3. 36. Menambahkan layanan



```
    while True:
        nama_layanan = input_teks("Nama layanan: ")
        nama_sudah_ada = any(
            layanan["nama_layanan"].lower() == nama_layanan.lower()
            for layanan in data_layanan
        )
        if nama_sudah_ada:
            print("Nama layanan sudah digunakan ⚠️")
        else:
            break
    harga = input_angka("Harga layanan: Rp", 1)
    layanan_baru = {
        "id_layanan": id_layanan,
        "nama_layanan": nama_layanan,
        "harga": str(harga)
    }
    data_layanan.append(layanan_baru)
    data_layanan.sort(key=kunci_id_layanan)
    try:
        simpan_csv_layanan(data_layanan)
        print("\n✅ Layanan berhasil ditambahkan. ✅")
    except OSError:
        data_layanan.remove(layanan_baru)
        print("Layanan gagal disimpan.")
    jeda()
```

Gambar 3. 37. Lanjutan menambah layanan

Selanjutnya, Fungsi `tambah_layanan()` digunakan untuk menambahkan layanan baru ke dalam sistem administrasi klinik. Pada awal fungsi, program membersihkan tampilan terminal, menampilkan judul halaman, dan membaca data layanan yang sudah tersimpan di file CSV. Selanjutnya, pengguna diminta memasukkan ID layanan baru dengan format empat karakter, yaitu diawali dengan `L00` dan diakhiri angka. Program akan memeriksa format ID serta memastikan bahwa ID tersebut belum pernah digunakan. Jika ID tidak sesuai atau sudah terdaftar, program akan menampilkan pesan peringatan dan meminta pengguna memasukkan ID kembali hingga benar.

Setelah ID berhasil divalidasi, program meminta pengguna memasukkan nama layanan. Pemeriksaan nama dilakukan menggunakan fungsi any() untuk memastikan nama layanan belum tersedia dalam data sebelumnya. Perintah .lower() digunakan agar perbandingan nama tidak membedakan penggunaan huruf kapital dan huruf kecil. Jika nama layanan sudah digunakan, program akan

menampilkan pesan peringatan dan meminta pengguna memasukkan nama lain. Namun, jika nama belum terdaftar, perulangan dihentikan menggunakan break sehingga proses dapat dilanjutkan ke tahap berikutnya.

Program melanjutkan proses penambahan layanan dengan meminta pengguna memasukkan harga layanan melalui fungsi input_angka(). Selanjutnya, data layanan baru disimpan sementara ke dalam variabel layanan_baru yang berbentuk dictionary dan berisi ID layanan, nama layanan, serta harga layanan. Perintah data_layanan.append(layanan_baru) digunakan untuk menambahkan data tersebut ke dalam daftar layanan. Setelah itu, data_layanan.sort(key=kunci_id_layanan) berfungsi mengurutkan data layanan berdasarkan ID agar tersusun secara teratur.

Tahap berikutnya adalah menyimpan data layanan ke dalam file CSV menggunakan fungsi simpan_csv_layanan(). Program menggunakan try untuk menjalankan proses penyimpanan dan menampilkan pesan bahwa layanan berhasil ditambahkan apabila proses berjalan dengan baik. Namun, jika terjadi kesalahan penyimpanan berupa OSError, program akan menjalankan except untuk menghapus data layanan yang sebelumnya ditambahkan menggunakan data_layanan.remove(layanan_baru) dan menampilkan pesan bahwa layanan gagal disimpan. Terakhir, fungsi jeda() digunakan untuk memberikan jeda sebelum program kembali ke tampilan berikutnya.

c. **Mengubah Layanan**

```
def ubah_layanan():
    bersihkan_layar()
    tampilkan_layanan()
    data_layanan = baca_csv_layanan()
    print("\n Ubah Data Layanan ")
    while True:
        id_layanan = input_teks("Masukkan ID layanan (L001): ").upper()
        if len(id_layanan) != 4 or not id_layanan.startswith("L00") or not id_layanan[3].isdigit():
            print("ID harus 4 karakter dengan format L00X ")
        else:
            break
    layanan = cari_layanan(id_layanan, data_layanan)
    if layanan is None:
        print("Layanan tidak ditemukan.")
        jeda()
        return
    print("\nData layanan yang dipilih:")
    print("Nama:", layanan["nama_layanan"])
    print("Harga:", format_rupiah(layanan["harga"]))
    nama_lama = layanan["nama_layanan"]
    harga_lama = layanan["harga"]
    while True:
        nama_baru = input_teks("Nama layanan baru: ")
        nama_duplikat = any(
            item["id_layanan"].lower() != layanan["id_layanan"].lower()
            and item["nama_layanan"].lower() == nama_baru.lower()
            for item in data_layanan
        )
        if nama_duplikat:
            print("Nama layanan sudah digunakan ")
        else:
            break
    layanan["nama_layanan"] = nama_baru
    layanan["harga"] = str(input_angka("Harga baru: Rp", 1))
    try:
        simpan_csv_layanan(data_layanan)
        print("\n Data layanan berhasil diubah. ")
    except OSError:
        layanan["nama_layanan"] = nama_lama
        layanan["harga"] = harga_lama
        print("Data layanan gagal disimpan. Perubahan dibatalkan.")
    jeda()
```

Gambar 3. 38. Mengubah layanan

Program juga memiliki fungsi ubah_layanan() yang digunakan untuk memperbarui informasi layanan yang sudah terdaftar. Admin terlebih dahulu melihat daftar layanan, kemudian memasukkan ID layanan yang ingin diubah. Program mencari layanan tersebut menggunakan fungsi cari_layanan(). Jika layanan ditemukan, admin dapat memasukkan nama dan harga baru. Sebelum perubahan dilakukan, program menyimpan nama dan harga lama sebagai cadangan.

Hal ini bertujuan agar informasi sebelumnya dapat dikembalikan apabila proses penyimpanan mengalami kesalahan. Setelah perubahan berhasil dilakukan, data layanan akan disimpan kembali ke file CSV sehingga informasi yang ditampilkan pada penggunaan berikutnya sudah menggunakan data terbaru.

d. **Menghapus Layanan**

```
def hapus_layanan():
    bersihkan_layar()
    tampilkan_layanan()
    data_layanan = baca_csv_layanan()
    print("\n Hapus Layanan ")
    id_layanan = input_teks("Masukkan ID layanan: ")
    layanan = cari_layanan(id_layanan, data_layanan)
    if layanan is None:
        print("Layanan tidak ditemukan.")
        jeda()
        return
    if not konfirmasi("Yakin ingin menghapus layanan " + layanan["nama_layanan"] + "?"):
        print("Penghapusan dibatalkan.")
        jeda()
        return
    indeks = data_layanan.index(layanan)
    data_layanan.remove(layanan)
    try:
        simpan_csv_layanan(data_layanan)
        print("Layanan berhasil dihapus dari daftar.")
    except (OSError, ValueError, csv.Error) as error:
        data_layanan.insert(indeks, layanan)
        print("Layanan gagal dihapus:", error)
    jeda()
```

Gambar 3. 39. Menghapus layanan

Selain menambahkan dan mengubah layanan, admin juga dapat menghapus layanan menggunakan fungsi hapus_layanan(). Pada proses ini, program menampilkan daftar layanan dan meminta admin memasukkan ID layanan yang ingin dihapus. Jika ID ditemukan, program menampilkan informasi layanan dan meminta konfirmasi sebelum melanjutkan penghapusan. Apabila admin menyetujui, data layanan dihapus menggunakan fungsi remove(), sedangkan posisi data sebelumnya disimpan menggunakan index().

Data yang sudah diperbarui kemudian disimpan ke dalam file CSV. Jika penyimpanan gagal, fungsi insert() digunakan untuk mengembalikan layanan ke posisi sebelumnya. Seluruh fungsi tersebut dihubungkan melalui menu_kelola_layanan() yang menyediakan pilihan untuk melihat, menambah, mengubah, menghapus layanan, dan kembali ke menu admin.

3.2.12. Pencatatan Transaksi, Pembuatan ID Transaksi dan Pembuatan Invoice

a. **Pembuatan ID Transaksi**

```
def buat_id_transaksi():
    data = baca_csv(FILE_TRANSAKSI, KOLOM_TRANSAKSI)
    tanggal = datetime.now().strftime("%Y%m%d")
    id_terpakai = [transaksi["id_transaksi"] for transaksi in data]

    while True:
        nomor = random.randint(1000, 9999)
        id_transaksi = "TRX" + tanggal + str(nomor)
        if id_transaksi not in id_terpakai:
            return id_transaksi
```

Gambar 3. 40. Pembuatan ID transaksi

Program membuat ID transaksi secara otomatis dengan menggabungkan kode TRX, tanggal transaksi, dan empat digit angka acak. Sistem juga memeriksa agar ID yang dibuat tidak sama dengan ID transaksi yang sudah pernah digunakan.

b. **Memasukkan ID Transaksi**

```
def simpan_transaksi(username, jenis, id_layanan="", nama_layanan="", jumlah=1,
                     nominal=0, saldo_awal=0, saldo_akhir=0, status="Berhasil"):
    data = baca_csv(FILE_TRANSAKSI, KOLOM_TRANSAKSI)
    data.append({
        "id_transaksi": buat_id_transaksi(),
        "waktu": datetime.now().strftime("%d-%m-%Y %H:%M:%S"),
        "username": username,
        "jenis": jenis,
        "id_layanan": id_layanan,
        "nama_layanan": nama_layanan,
        "jumlah": str(jumlah),
        "nominal": str(nominal),
        "saldo_awal": str(saldo_awal),
        "saldo_akhir": str(saldo_akhir),
        "status": status
    })
    simpan_csv(FILE_TRANSAKSI, KOLOM_TRANSAKSI, data)
    return data[-1]
```

Gambar 3. 41. Memasukkan ID Transaksi

Setelah ID transaksi dibuat, sistem memasukkannya secara otomatis ke dalam database transaksi melalui fungsi simpan_transaksi(). Data yang disimpan meliputi tanggal, username, jenis transaksi, layanan, jumlah, nominal, saldo awal, saldo akhir, dan status transaksi.

c. **Menghitung Total Biaya Layanan**

```
layanan = cari_layanan(input_teks("\nMasukkan ID layanan: "), data_layanan)
if layanan is None:
    print("ID layanan tidak ditemukan ⚠️")
    jeda()
    return
jumlah = 1
total = int(layanan["harga"])
data_users = baca_csv(FILE_USERS, KOLOM_USERS)
tersimpan = cari_user(user["username"], data_users)
if tersimpan is None:
    print("Akun tidak ditemukan.")
    jeda()
    return
saldo_awal = int(tersimpan["saldo"])
if saldo_awal < total:
    print("❌ Saldo tidak mencukupi.")
    print("Total pembayaran:", format_rupiah(total))
    print("Saldo saat ini  :", format_rupiah(saldo_awal))
    jeda()
    return
tampilkan_judul("Ringkasan Pembayaran", "🧾")
tampilkan_tabel("🧾 Rincian Tagihan 🧾", ["Layanan", "Harga", "Jumlah", "Total"],
                [[layanan["nama_layanan"], format_rupiah(layanan["harga"]),
                jumlah, format_rupiah(total)]])
if not konfirmasi("Lanjutkan pembayaran?"):
    print("Pembayaran dibatalkan.")
    jeda()
    return
saldo_akhir = saldo_awal - total
tersimpan["saldo"] = str(saldo_akhir)
try:
    simpan_csv(FILE_USERS, KOLOM_USERS, data_users)
    transaksi = simpan_transaksi(user["username"], "PEMBAYARAN_LAYANAN",
                                 layanan["id_layanan"], layanan["nama_layanan"],
                                 jumlah, total, saldo_awal, saldo_akhir)
```

Gambar 3. 42. Menghitung total biaya layanan

Program mengambil harga layanan berdasarkan layanan yang dipilih oleh pengguna. Harga tersebut digunakan sebagai total biaya pembayaran. Selanjutnya, sistem memeriksa kecukupan saldo pengguna sebelum transaksi diproses dan menghitung saldo akhir setelah pembayaran.

d. **Membuat Invoice Transaksi**

```
def tampilkan_invoice(transaksi):
    garis = "=" * 44
    waktu_transaksi = transaksi.get("waktu") or datetime.now().strftime("%d-%m-%Y %H:%M:%S")
    jenis = transaksi.get("jenis", "-").replace("_", " ").title()
    status = transaksi.get("status", "Tidak diketahui")
    ikon_status = "✅" if status.lower() == "berhasil" else "⚠️"
    tampilkan_judul("INVOICE KLINIK", "🧾", 44)
    print()
    print("📌 INFORMASI TRANSAKSI")
    print(garis)
    print(f"🆔 No. Transaksi : {transaksi.get('id_transaksi', '-')}")
    print(f"📅 Tanggal       : {waktu_transaksi}")
    print(f"👤 Pengguna      : {transaksi.get('username', '-')}")
    print(f"📂 Jenis         : {jenis}")
    if transaksi.get("nama_layanan"):
        print()
        print("🧾 RINCIAN LAYANAN")
        print(garis)
        print(f"💉 Layanan       : {transaksi.get('nama_layanan', '-')}")
        print(f"🔢 Jumlah        : {transaksi.get('jumlah', '1')}")
    print()
    print("💰 RINCIAN SALDO")
    print(garis)
    print(f"💵 Nominal       : {format_rupiah(transaksi.get('nominal', 0))}")
    print(f"⬅️ Saldo awal    : {format_rupiah(transaksi.get('saldo_awal', 0))}")
    print(f"➡️ Saldo akhir   : {format_rupiah(transaksi.get('saldo_akhir', 0))}")
    print(f"{ikon_status} Status        : {status}")
    print(garis)
    print("💙 Terima kasih telah menggunakan layanan kami.")
```

Gambar 3. 43. Membuat invoice transaksi

Setelah transaksi berhasil disimpan, sistem menampilkan invoice melalui fungsi tampilkan_invoice(). Invoice berisi ID transaksi, tanggal transaksi, username, rincian layanan, jumlah, total biaya, saldo awal, saldo akhir, dan status transaksi.

3.2.13. Top-up Saldo

a. **Memasukkan Nominal Top-up**

```
def topup_saldo(user):
    bersihkan_layar()
    tampilkan_judul("Top-up Saldo", "💳")
    nominal = input_angka("Nominal top-up (Rp): ", 1000)
    if nominal > 10000000:
        print("Nominal maksimal top-up adalah Rp10.000.000.")
        jeda()
        return
```

Gambar 3. 44. Memasukkan nominal top up

Fungsi topup_saldo(user) digunakan untuk memungkinkan pengguna melakukan pengisian saldo ke dalam akun masing-masing. Pada awal proses, program membersihkan layar dan menampilkan judul menu top-up. Selanjutnya, pengguna diminta memasukkan nominal

pengisian saldo melalui fungsi input_angka() dengan batas minimal Rp1.000.

b. **Memvalidasi Nominal Top-up**

```
nominal = input_angka("Nominal top-up (Rp): ", 1000)
if nominal > 10000000:
    print("Nominal maksimal top-up adalah Rp10.000.000.")
    jeda()
    return
if not konfirmasi("Konfirmasi top-up " + format_rupiah(nominal) + "?"):
    print("Top-up dibatalkan.")
    jeda()
    return
```

Gambar 3. 45. Validasi nominal top up

Program kemudian melakukan pemeriksaan untuk memastikan bahwa nominal yang dimasukkan tidak melebihi batas maksimal Rp10.000.000. Jika nominal tidak memenuhi ketentuan, program akan menampilkan pesan kesalahan dan menghentikan proses top-up.

c. **Memperbarui Saldo Pengguna**

```
saldo_awal = int(tersimpan["saldo"])
saldo_akhir = saldo_awal + nominal
tersimpan["saldo"] = str(saldo_akhir)
try:
    simpan_csv(FILE_USERS, KOLOM_USERS, data_users)
    transaksi = simpan_transaksi(user["username"], "TOP_UP", nominal=nominal,
                                 saldo_awal=saldo_awal, saldo_akhir=saldo_akhir)
    user["saldo"] = str(saldo_akhir)
```

Gambar 3. 46. Memperbarui saldo pengguna

Apabila nominal sudah sesuai, program meminta konfirmasi kepada pengguna sebelum melanjutkan transaksi. Jika pengguna menyetujui, sistem membaca data akun dari file users.csv dan mencari akun yang sedang digunakan melalui fungsi cari_user(). Setelah akun ditemukan, program mengambil saldo awal menggunakan int() dan menjumlahkannya dengan nominal top-up untuk memperoleh saldo akhir. Saldo yang baru kemudian diperbarui pada data pengguna dan disimpan kembali ke dalam file CSV.

Program juga mencatat aktivitas tersebut menggunakan fungsi simpan_transaksi() dengan jenis transaksi TOP_UP. Setelah seluruh proses berhasil, saldo pada data pengguna yang sedang aktif diperbarui dan invoice ditampilkan sebagai bukti transaksi. Jika terjadi

kesalahan ketika penyimpanan, program akan mencoba mengembalikan saldo ke jumlah sebelumnya agar perubahan yang gagal tidak tetap diterapkan.

3.2.14. Pembayaran Layanan Klinik

a. **Memilih Layanan yang Akan Dibayar**

```
tampilkan_layanan()
data_layanan = baca_csv_layanan()
if not data_layanan:
    print("Belum ada layanan yang terdaftar.")
    jeda()
    return
layanan = cari_layanan(input_teks("\nMasukkan ID layanan: "), data_layanan)
if layanan is None:
    print("ID layanan tidak ditemukan ⚠")
    jeda()
    return
```

Gambar 3. 47. Memilih layanan yang akan dibayar

Fungsi bayar_layanan(user) digunakan untuk memungkinkan pengguna melakukan pembayaran terhadap layanan kesehatan yang tersedia di klinik menggunakan saldo akun. Pada awal proses, program membersihkan layar, menampilkan judul pembayaran, dan memperlihatkan daftar layanan yang tersedia melalui fungsi tampilkan_layanan(). Selanjutnya, program membaca data layanan dari file CSV dan memeriksa apakah terdapat layanan yang dapat dipilih. Jika daftar layanan kosong, proses pembayaran tidak dapat dilanjutkan.

Apabila layanan tersedia, pengguna diminta memasukkan ID layanan yang ingin dibeli. ID tersebut kemudian dicari menggunakan fungsi cari_layanan(). Jika ID tidak ditemukan, program menampilkan pemberitahuan bahwa layanan tidak tersedia. Namun, jika layanan ditemukan, program mengambil harga layanan dan menetapkan jumlah pembelian sebanyak satu.

b. **Memeriksa Kecukupan Saldo**

```
saldo_awal = int(tersimpan["saldo"])
if saldo_awal < total:
    print("❌ Saldo tidak mencukupi.")
    print("Total pembayaran:", format_rupiah(total))
    print("Saldo saat ini  :", format_rupiah(saldo_awal))
    jeda()
    return
```

Gambar 3. 48. Memeriksa kecukupan saldo

Setelah itu, sistem membaca data pengguna dan mencari akun yang sedang melakukan transaksi. Program kemudian mengambil saldo awal pengguna dan membandingkannya dengan total harga layanan. Apabila saldo lebih kecil daripada total pembayaran, program menampilkan informasi bahwa saldo tidak mencukupi beserta nominal tagihan dan saldo yang dimiliki pengguna.

c. **Memproses Pembayaran dan Memperbarui Saldo**

```
saldo_akhir = saldo_awal - total
tersimpan["saldo"] = str(saldo_akhir)
try:
    simpan_csv(FILE_USERS, KOLOM_USERS, data_users)
    transaksi = simpan_transaksi(user["username"], "PEMBAYARAN_LAYANAN",
                                 layanan["id_layanan"], layanan["nama_layanan"],
                                 jumlah, total, saldo_awal, saldo_akhir)
    user["saldo"] = str(saldo_akhir)
    print("\n┌──────────────────────────────────────┐")
    print("│  ✅ PEMBAYARAN BERHASIL ✅           │")
    print("└──────────────────────────────────────┘")
    tampilkan_invoice(transaksi)
```

Gambar 3. 49. Memproses pembayaran dan memperbarui saldo

Jika saldo mencukupi, program menampilkan rincian tagihan menggunakan PrettyTable yang berisi nama layanan, harga, jumlah, dan total pembayaran. Pengguna kemudian diminta memberikan konfirmasi untuk melanjutkan pembayaran. Apabila pengguna menyetujui, program mengurangi saldo awal dengan total pembayaran untuk memperoleh saldo akhir. Saldo yang sudah diperbarui kemudian disimpan ke dalam file CSV dan aktivitas pembayaran dicatat menggunakan fungsi simpan_transaksi() dengan jenis transaksi PEMBAYARAN_LAYANAN. Setelah pembayaran berhasil, program memperbarui saldo pada akun yang sedang aktif dan

menampilkan invoice sebagai bukti pembayaran. Jika terjadi kesalahan dalam proses penyimpanan, program akan mencoba mengembalikan saldo ke nilai sebelumnya.

3.2.15. Riwayat Transaksi

a. Menampilkan Riwayat Transaksi Pengguna

```
def tampilkan_riwayat(username=None):
    data = baca_csv(FILE_TRANSAKSI, KOLOM_TRANSAKSI)
    if username is not None:
        data = [x for x in data if x["username"].lower() == username.lower()]
    data.reverse()
    baris = [[x["id_transaksi"], x["waktu"], x["username"],
              x["jenis"].replace("_", " ").title(),
              x["nama_layanan"] or "-", format_rupiah(x["nominal"]), x["status"]]
             for x in data]
    tampilkan_tabel("📄 RIWAYAT TRANSAKSI 📄",
                    ["ID Transaksi", "Waktu", "Username", "Jenis", "Layanan", "Nominal", "Status"],
                    baris if baris else [["-", "-", "-", "-", "Belum ada transaksi", "-", "-"]])
```

Gambar 3. 50. Menampilkan riwayat traksaksi pengguna

Fungsi tampilkan_riwayat(username=None) digunakan untuk menampilkan catatan transaksi yang telah dilakukan oleh pengguna. Pada awal proses, program membaca seluruh data transaksi dari file transaksi.csv. Fungsi ini memiliki parameter username yang bersifat opsional sehingga dapat digunakan untuk menampilkan seluruh transaksi atau hanya transaksi milik pengguna tertentu. Jika username diberikan, program menggunakan list comprehension untuk menyaring data berdasarkan username yang sesuai. Namun, apabila parameter username tidak diberikan, seluruh transaksi akan ditampilkan. Setelah itu, fungsi reverse() digunakan untuk membalik urutan data agar transaksi terbaru muncul pada bagian paling atas.

b. **Menampilkan Detail Transaksi**

```
def tampilkan_invoice(transaksi):
    garis = "" * 44
    waktu_transaksi = transaksi.get("waktu") or datetime.now().strftime("%d-%m-%Y %H:%M:%S")
    jenis = transaksi.get("jenis", "-").replace("_", " ").title()
    status = transaksi.get("status", "Tidak diketahui")
    ikon_status = "" if status.lower() == "berhasil" else ""
    tampilkan_judul("INVOICE KLINIK", "", 44)
    print()
    print("" INFORMASI TRANSAKSI")
    print(garis)
    print(f"ID No. Transaksi : {transaksi.get('id_transaksi', '-')}")
    print(f" Tanggal       : {waktu_transaksi}")
    print(f" Pengguna      : {transaksi.get('username', '-')}")
    print(f"  Jenis         : {jenis}")
    if transaksi.get("nama_layanan"):
        print()
        print(" RINCIAN LAYANAN")
        print(garis)
        print(f" Layanan       : {transaksi.get('nama_layanan', '-')}")
        print(f" Jumlah        : {transaksi.get('jumlah', '1')}")
    print()
    print(" RINCIAN SALDO")
    print(garis)
    print(f" Nominal       : {format_rupiah(transaksi.get('nominal', 0))}")
    print(f"  Saldo awal    : {format_rupiah(transaksi.get('saldo_awal', 0))}")
    print(f"  Saldo akhir   : {format_rupiah(transaksi.get('saldo_akhir', 0))}")
    print(f"{ikon_status} Status        : {status}")
    print(garis)
    print(" Terima kasih telah menggunakan layanan kami.")
```

Gambar 3. 51. Menampilkan detail transaksi

Program kemudian menyusun informasi transaksi menggunakan list comprehension yang berisi ID transaksi, waktu transaksi, username, jenis transaksi, nama layanan, nominal, dan status. Metode replace() digunakan untuk mengganti tanda garis bawah pada jenis transaksi menjadi spasi, sedangkan title() digunakan untuk membuat huruf awal setiap kata menjadi kapital. Nama layanan yang kosong akan diganti dengan tanda strip menggunakan operator or. Nominal transaksi juga diubah menjadi format rupiah melalui fungsi format_rupiah().

c. **Menyimpan Data Riwayat Transaksi**

```
def simpan_transaksi(username, jenis, id_layanan="", nama_layanan="", jumlah=1,
                     nominal=0, saldo_awal=0, saldo_akhir=0, status="Berhasil"):
    data = baca_csv(FILE_TRANSAKSI, KOLOM_TRANSAKSI)
    data.append({
        "id_transaksi": buat_id_transaksi(),
        "waktu": datetime.now().strftime("%d-%m-%Y %H:%M:%S"),
        "username": username,
        "jenis": jenis,
        "id_layanan": id_layanan,
        "nama_layanan": nama_layanan,
        "jumlah": str(jumlah),
        "nominal": str(nominal),
        "saldo_awal": str(saldo_awal),
        "saldo_akhir": str(saldo_akhir),
        "status": status
    })
    simpan_csv(FILE_TRANSAKSI, KOLOM_TRANSAKSI, data)
    return data[-1]
```

Gambar 3. 52. Menyimpan data riwayat transaksi

Setelah seluruh informasi tersusun, program menampilkannya menggunakan fungsi tampilkan_tabel(). Jika belum terdapat transaksi, program akan menampilkan pemberitahuan bahwa belum ada transaksi yang tercatat.

3.2.16. Pengelolaan Pengguna oleh Admin

a. **Menampilkan Daftar Pengguna**

```
data = baca_csv(FILE_USERS, KOLOM_USERS)
tampilkan_tabel("👥 Daftar Pengguna 👥",
                ["Username", "Nama", "Role", "Saldo"],
                [[u["username"], u["nama"], u["role"], format_rupiah(u["saldo"])] for u in data])
```

Gambar 3. 53. Menampilkan daftar pengguna

Fungsi menu_kelola_pengguna() digunakan oleh admin untuk melihat dan mengelola akun pengguna yang telah terdaftar dalam sistem. Pada awal fungsi, program membersihkan layar, menampilkan judul menu, dan membaca data pengguna dari File CSV.

b. **Melihat Informasi Pengguna**

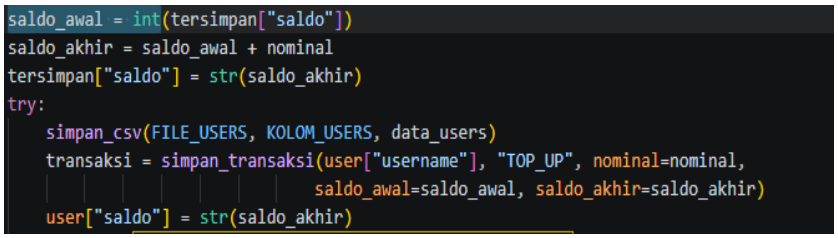
```
data = baca_csv(FILE_USERS, KOLOM_USERS)
tampilkan_tabel("👥 Daftar Pengguna 👥",
                ["Username", "Nama", "Role", "Saldo"],
                [[u["username"], u["nama"], u["role"], format_rupiah(u["saldo"])] for u in data])
```

Gambar 3. 54. Melihat informasi pengguna

Seluruh informasi pengguna kemudian ditampilkan dalam bentuk tabel yang berisi username, nama lengkap, role, dan saldo.

Setelah itu, admin diberikan pilihan untuk mengubah role pengguna, menghapus akun pengguna, atau kembali ke menu sebelumnya.

c. **Mengelola Data Pengguna**



```
saldo_awal = int(tersimpan["saldo"])
saldo_akhir = saldo_awal + nominal
tersimpan["saldo"] = str(saldo_akhir)
try:
    simpan_csv(FILE_USERS, KOLOM_USERS, data_users)
    transaksi = simpan_transaksi(user["username"], "TOP_UP", nominal=nominal,
                                 saldo_awal=saldo_awal, saldo_akhir=saldo_akhir)
    user["saldo"] = str(saldo_akhir)
```

Gambar 3. 55. Mengelola data pengguna

Ketika admin memilih menu untuk mengubah role, program meminta username akun yang ingin diperbarui. Username tersebut kemudian dicari menggunakan fungsi cari_user(). Jika akun tidak ditemukan, program menampilkan pemberitahuan bahwa pengguna tidak tersedia. Program juga memberikan perlindungan terhadap akun admin utama agar role-nya tidak dapat diubah. Jika akun dapat diubah, admin diminta memasukkan role baru yang hanya diperbolehkan berupa admin atau user. Setelah role dinyatakan valid, program meminta konfirmasi perubahan. Jika admin menyetujui, role lama disimpan terlebih dahulu sebagai cadangan, kemudian diganti dengan role baru dan disimpan ke file CSV. Apabila penyimpanan gagal, program mengembalikan role ke nilai sebelumnya.

Selain mengubah role, admin juga dapat menghapus akun pengguna melalui menu yang tersedia. Program meminta username akun yang ingin dihapus, kemudian mencari data tersebut dalam daftar pengguna. Akun yang tidak ditemukan tidak dapat dihapus, sedangkan akun admin utama dilindungi agar tidak terhapus. Jika akun ditemukan dan admin menyetujui konfirmasi penghapusan, program menyimpan posisi data menggunakan index() kemudian menghapus akun menggunakan remove(). Data yang sudah diperbarui selanjutnya disimpan kembali ke file CSV. Apabila penyimpanan gagal, akun akan dikembalikan ke daftar menggunakan insert(). Dengan adanya

fungsi ini, admin dapat mengatur hak akses serta mengelola akun yang terdaftar dalam sistem.

3.2.17. Laporan Transaksi untuk Admin

a. **Menampilkan Data Transaksi**

```
def laporan_transaksi():
    data = baca_csv(FILE_TRANSAKSI, KOLOM_TRANSAKSI)
    berhasil = [x for x in data if x["status"] == "Berhasil"]
    total_topup = sum(int(x["nominal"]) for x in berhasil if x["jenis"] == "TOP_UP")
    total_bayar = sum(int(x["nominal"]) for x in berhasil if x["jenis"] == "PEMBAYARAN_LAYANAN")
```

Gambar 3. 56. Menampilkan data transaksi

Fungsi laporan_transaksi() digunakan untuk memberikan informasi mengenai keseluruhan aktivitas transaksi yang terjadi dalam sistem klinik. Program terlebih dahulu membaca seluruh data transaksi dari File CSV, kemudian menggunakan list comprehension untuk memisahkan transaksi yang memiliki status berhasil.

b. **Menghitung Total Transaksi**

```
total_topup = sum(int(x["nominal"]) for x in berhasil if x["jenis"] == "TOP_UP")
total_bayar = sum(int(x["nominal"]) for x in berhasil if x["jenis"] == "PEMBAYARAN_LAYANAN")
```

Gambar 3. 57. Menghitung total transaksi

Setelah itu, program menghitung total nominal top-up dengan menggunakan fungsi sum() terhadap transaksi yang memiliki jenis TOP_UP. Proses yang sama dilakukan untuk menghitung total pembayaran layanan dengan mengambil transaksi yang memiliki jenis PEMBAYARAN_LAYANAN. Fungsi int() digunakan untuk mengubah nominal yang awalnya berbentuk teks menjadi angka sehingga dapat dijumlahkan.

c. **Menyajikan Laporan Transaksi**

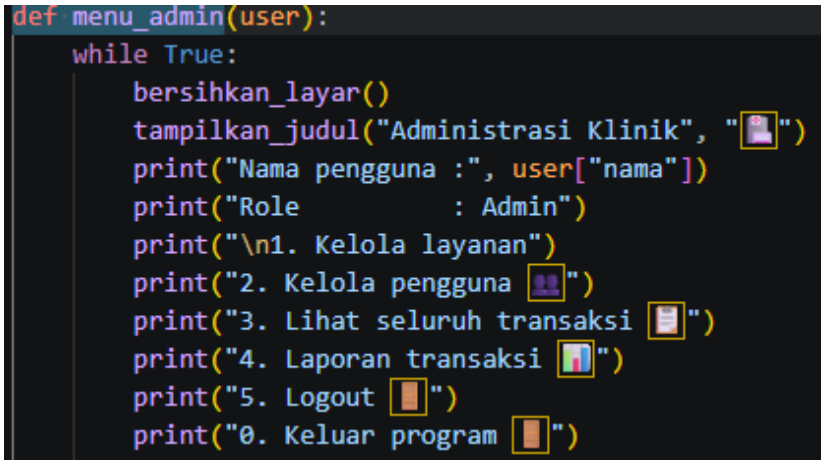
```
tampilkan_judul("Laporan Transaksi", "📊")
tampilkan_tabel("📋 Ringkasan 📋", ["Indikator", "Jumlah"],
                [["Total transaksi", len(data)],
                 ["Transaksi berhasil", len(berhasil)],
                 ["Total top-up", format_rupiah(total_topup)],
                 ["Total pembayaran layanan", format_rupiah(total_bayar)]])
layanan_terjual = {}
for x in berhasil:
    if x["jenis"] == "PEMBAYARAN_LAYANAN" and x["nama_layanan"]:
        layanan_terjual[x["nama_layanan"]] = layanan_terjual.get(x["nama_layanan"], 0) + 1
if layanan_terjual:
    terlaris = max(layanan_terjual, key=layanan_terjual.get); tampilkan_tabel("🏆 Layanan Terlaris 🏆", ["Indikator", "Jumlah"], [["Nama layanan", terlaris], ["Jumlah transaksi", layanan_terjual[terlaris]]])
else:
    print("Belum ada transaksi layanan yang berhasil.")
tampilkan_riwayat()
```

Gambar 3. 58. Menyajikan laporan transaksi

Program menampilkan laporan transaksi yang tersimpan, meliputi jumlah seluruh transaksi, transaksi berhasil, total top-up, dan total pembayaran layanan. Program juga menampilkan layanan yang paling banyak digunakan beserta jumlah transaksinya. Selain itu, admin dapat melihat riwayat transaksi untuk memantau aktivitas pembayaran dan top-up yang terjadi di dalam sistem klinik.

3.2.18. Menu Admin

a. **Menampilkan Menu Admin**



Gambar 3. 59. Menampilkan menu admin

Fungsi menu_admin(user) merupakan menu utama yang diberikan kepada pengguna dengan hak akses sebagai admin. Setelah admin berhasil login, program akan menampilkan judul administrasi klinik beserta nama dan role pengguna. Menu ini dibuat menggunakan perulangan while True agar admin dapat memilih berbagai fitur secara berulang sampai melakukan logout atau keluar dari program.

b. **Memilih Fitur Administrasi**

```
pilihan = input_teks("\nPilih menu: ")
if pilihan == "1":
    menu_kelola_layanan()
elif pilihan == "2":
    menu_kelola_pengguna()
elif pilihan == "3":
    bersihkan_layar()
    tampilkan_riwayat()
    jeda()
elif pilihan == "4":
    bersihkan_layar()
    laporan_transaksi()
    jeda()
```

Gambar 3. 60. Memilih menu administrasi

Admin memiliki akses untuk mengelola layanan klinik, mengelola pengguna, melihat seluruh riwayat transaksi, dan melihat laporan transaksi. Setiap pilihan yang dimasukkan admin akan diperiksa menggunakan percabangan if, elif, dan else.

c. **Menjalankan Fungsi Admin**

```
elif pilihan == "5":
    print("✋ Logout berhasil. ✋")
    jeda()
    break
elif pilihan == "0":
    if konfirmasi("Yakin ingin keluar dari program?"):
        return False
else:
    print("Pilihan tidak tersedia ⚠")
    jeda()
turn True
```

Gambar 3. 61. Menjalankan fungsi admin

Jika admin memilih menu pertama, program akan menjalankan Fungsi menu_kelola_layanan() yang menyediakan fitur untuk melihat, menambah, mengubah, dan menghapus layanan. Jika memilih menu kedua, program akan menjalankan fungsi menu_kelola_pengguna() untuk mengatur akun pengguna. Pilihan ketiga digunakan untuk

menampilkan seluruh riwayat transaksi, sedangkan pilihan keempat digunakan untuk membuka laporan transaksi.

Apabila admin memilih logout, perulangan akan dihentikan menggunakan break sehingga pengguna dapat kembali ke menu autentikasi. Sementara itu, jika admin memilih keluar dari program, sistem akan meminta konfirmasi terlebih dahulu. Jika disetujui, fungsi akan mengembalikan nilai false sebagai tanda bahwa program harus dihentikan.

3.2.19. Menu User

a. **Menampilkan Menu Pengguna**

```
def menu_user(user):
    while True:
        bersihkan_layar()
        data_users = baca_csv(FILE_USERS, KOLOM_USERS)
        user_tersimpan = cari_user(user["username"], data_users)
        if user_tersimpan is None:
            print("Data pengguna tidak ditemukan.")
            jeda()
            return True
        user["saldo"] = user_tersimpan["saldo"]
        tampilkan_judul("Layanan Klinik", "")
        print("Nama pengguna :", user["nama"])
        print("Username      :", user["username"])
        print("Saldo         :", format_rupiah(user["saldo"]))
```

Gambar 3. 62. Menampilkan menu pengguna

Fungsi menu_user(user) merupakan menu utama yang ditampilkan kepada pengguna biasa setelah berhasil melakukan login. Pada awal fungsi, program membersihkan layar dan membaca kembali data pengguna dari file CSV. Langkah ini dilakukan untuk memastikan informasi akun yang digunakan selalu sesuai dengan data terakhir yang tersimpan, terutama informasi saldo. Program kemudian mencari akun berdasarkan username yang sedang login menggunakan fungsi cari_user(). Jika akun tidak ditemukan, sistem akan menampilkan pesan kesalahan dan mengembalikan pengguna ke alur

utama. Namun, jika akun ditemukan, saldo pada data pengguna yang sedang aktif akan diperbarui berdasarkan informasi yang tersimpan dalam file CSV. Setelah data akun diperbarui, program menampilkan nama pengguna, username, dan jumlah saldo yang dimiliki.

b. **Memilih Fitur Layanan**

```
print("\n1. Lihat layanan ")
print("2. Top-up saldo ")
print("3. Bayar layanan ")
print("4. Lihat riwayat transaksi ")
print("5. Logout ")
print("0. Keluar program ")
pilihan = input_teks("\nPilih menu: ")
```

Gambar 3. 63. Memilih fitur layanan

Selanjutnya, pengguna diberikan pilihan untuk melihat daftar layanan klinik, melakukan top-up saldo, melakukan pembayaran layanan, melihat riwayat transaksi pribadi, logout, atau keluar dari program. Setiap pilihan akan menjalankan fungsi yang sesuai.

c. **Menjalankan Fungsi Pengguna**

```
    if pilihan == "1":
        bersihkan_layar()
        tampilkan_layanan()
        jeda()
    elif pilihan == "2":
        topup_saldo(user)
    elif pilihan == "3":
        bayar_layanan(user)
    elif pilihan == "4":
        bersihkan_layar()
        tampilkan_riwayat(user["username"])
        jeda()
    elif pilihan == "5":
        print(" Logout berhasil. ")
        jeda()
        break
    elif pilihan == "0":
        if konfirmasi("Yakin ingin keluar dari program?"):
            return False
    else:
        print("Pilihan tidak tersedia ")
        jeda()
return True
```

Gambar 3. 64. Menjalankan fungsi pengguna

Jika pengguna memilih melihat layanan, program menjalankan fungsi tampilkan_layanan(). Jika memilih top-up, program menjalankan fungsi topup_saldo(user), sedangkan pilihan pembayaran akan menjalankan fungsi bayar_layanan(user). Untuk melihat riwayat transaksi, program memanggil fungsi tampilkan_riwayat() dengan menyertakan username pengguna sehingga transaksi yang ditampilkan hanya milik akun tersebut.

Jika pengguna memilih logout, program akan menghentikan perulangan menu menggunakan break sehingga pengguna dapat kembali ke menu autentikasi. Sementara itu, jika pengguna memilih keluar, program akan meminta konfirmasi terlebih dahulu. Apabila pengguna menyetujui, fungsi mengembalikan nilai false agar sistem mengetahui bahwa program harus dihentikan. Dengan adanya menu ini, pengguna dapat melakukan transaksi secara mandiri tanpa memiliki akses untuk mengubah data layanan maupun mengelola akun pengguna lain.

3.2.20. Menu Autentikasi

a. **Proses Login Pengguna**

```
def menu_autentikasi():
    while True:
        bersihkan_layar()
        tampilkan_judul("Administrasi Transaksi Klinik", "")
        print("1. Login ")
        print("2. Registrasi pengguna ")
        print("0. Keluar program ")
        pilihan = input_teks("\nPilih menu: ")
        if pilihan == "1":
            user = login()
            if user is not None:
                return user
```

Gambar 3. 65. Proses login pengguna

Fungsi menu_autentikasi() merupakan tampilan awal yang digunakan untuk menghubungkan pengguna dengan proses login dan

registrasi. Ketika fungsi dijalankan, program membersihkan layar dan menampilkan judul Administrasi Transaksi Klinik. Selanjutnya, pengguna diberikan tiga pilihan, yaitu login, registrasi pengguna, dan keluar dari program. Menu tersebut menggunakan perulangan while True agar pengguna dapat memilih kembali apabila belum berhasil masuk ke dalam sistem. Jika pengguna memilih login, program akan memanggil Fungsi login() untuk memeriksa username dan password.

b. **Validasi Username dan Password**

```
user = cari_user(username, data_users)
if user is not None:
    if user["password_hash"] == hash_password(password):
        if user["role"] not in ROLE_PENGGUNA:
            print("Role akun tidak valid.")
            jeda()
            return None
```

Gambar 3. 66. Validasi username dan password

Apabila proses login berhasil, fungsi tersebut akan mengembalikan data akun yang kemudian diteruskan melalui return user.

c. **Pembagian Hak Akses Admin dan User**

```
if user["role"] not in ROLE_PENGGUNA:
    print("Role akun tidak valid.")
    jeda()
    return None
```

Gambar 3. 67. Pembagian hak akses pengguna

Data tersebut nantinya digunakan untuk menentukan menu yang sesuai dengan hak akses pengguna. Jika pengguna memilih registrasi, program akan menjalankan fungsi registrasi() agar pengguna dapat membuat akun baru. Sementara itu, jika pengguna memilih keluar, program akan meminta konfirmasi terlebih dahulu. Apabila pengguna menyetujui, fungsi akan mengembalikan nilai None sebagai tanda bahwa pengguna tidak ingin melanjutkan penggunaan program.

3.2.21. Fungsi Utama Program (Main)

a. **Memulai Program**

```
def main():
    try:
        inisialisasi_data()
```

Gambar 3. 68. Memulai program

Fungsi main() merupakan bagian utama yang mengendalikan seluruh proses dalam program Administrasi Transaksi Klinik. Pada awal fungsi, terdapat blok try yang digunakan untuk menangani kesalahan selama program dijalankan. Kemudian, program memanggil Fungsi inisialisasi_data() untuk memastikan file pengguna dan layanan sudah tersedia serta memiliki data awal yang dibutuhkan.

b. **Menampilkan Menu Utama**

```
def main():
    try:
        inisialisasi_data()
        while True:
            user = menu_autentikasi()
            if user is None:
                bersihkan_layar()
                print(
                    "Terima kasih telah menggunakan "
                    "Administrasi Transaksi Klinik."
                )
                break
            if user["role"] == "admin":
                lanjut = menu_admin(user)
            else:
                lanjut = menu_user(user)
            if not lanjut:
                bersihkan_layar()
                print(
                    "Terima kasih telah menggunakan "
                    "Administrasi Transaksi Klinik."
                )
                break
    except FileNotFoundError as error:
        print("File tidak ditemukan:", error)
    except ValueError as error:
        print("Terjadi kesalahan pada data:", error)
    except OSError as error:
        print("Terjadi kesalahan saat mengakses file:", error)
```

Gambar 3. 69. Menampilkan menu utama

Setelah proses inisialisasi selesai, program menjalankan perulangan while True dan memanggil fungsi menu_autentikasi() untuk menampilkan pilihan login, registrasi, atau keluar dari program.

c. **Mengatur Alur Jalannya Program**

```
if user is None:
    bersihkan_layar()
    print(
        "Terima kasih telah menggunakan "
        "Administrasi Transaksi Klinik."
    )
    break
if user["role"] == "admin":
    lanjut = menu_admin(user)
else:
    lanjut = menu_user(user)
if not lanjut:
    bersihkan_layar()
    print(
        "Terima kasih telah menggunakan "
        "Administrasi Transaksi Klinik."
    )
    break
```

Gambar 3. 70. Mengatur alur jalannya program

Jika pengguna memilih keluar atau tidak berhasil masuk ke dalam sistem, Fungsi menu_autentikasi() akan mengembalikan nilai None. Kondisi tersebut diperiksa menggunakan if user is None. Apabila benar, program akan membersihkan layar, menampilkan ucapan terima kasih, dan menghentikan perulangan menggunakan break. Namun, jika pengguna berhasil login, program akan memeriksa nilai role pada akun tersebut. Jika role bernilai admin, sistem akan menjalankan fungsi menu_admin(user), sedangkan jika role bernilai user, sistem akan menjalankan fungsi menu_user(user). Dengan demikian, fungsi utama bertugas mengarahkan pengguna ke menu yang sesuai berdasarkan hak akses masing-masing.

Selain mengatur jalannya program, fungsi main() juga memiliki beberapa blok except untuk menangani kesalahan yang mungkin terjadi. FileNotFoundError digunakan untuk menangani kondisi ketika file yang dibutuhkan tidak ditemukan, ValueError digunakan untuk menangani kesalahan yang berkaitan dengan nilai atau format data, sedangkan OSError digunakan untuk menangani kesalahan yang terjadi ketika program mengakses atau menyimpan file. Penanganan kesalahan tersebut bertujuan agar program dapat memberikan informasi yang jelas apabila terjadi masalah selama proses berlangsung.

3.2.22. Menjalankan Program

a. **Persiapan File dan Data**



Gambar 3. 71. Persiapan file data

Pada bagian paling akhir kode terdapat perintah if __name__ == "__main__": main(). Perintah tersebut digunakan untuk memastikan bahwa fungsi main() hanya dijalankan ketika file Python dieksekusi secara langsung.

b. **Proses Eksekusi Program**



Gambar 3. 72. Proses eksekusi program

Dalam Python, variabel khusus __name__ akan memiliki nilai "__main__" apabila file dijalankan secara langsung. Oleh karena itu, kondisi if akan memeriksa nilai tersebut sebelum memanggil fungsi main(). Ketika kondisi terpenuhi, seluruh proses program akan dimulai dari fungsi utama, kemudian dilanjutkan ke fungsi-fungsi lainnya sesuai dengan pilihan pengguna.

c. **Pengujian Fungsi Program**

```
def main():
    try:
        inisialisasi_data()
        while True:
            user = menu_autentikasi()
```

Gambar 3. 73. Pengujian fungsi program

Bagian ini menjadi titik awal yang menghubungkan seluruh fungsi sehingga program dapat berjalan secara terstruktur.

3.3 Source Code

Tabel 3. 1. Source Code

```
import csv
import os
import time
import random
import hashlib
from datetime import datetime
from prettytable import PrettyTable
from pwinput import pwinput
FOLDER_PROGRAM = os.path.dirname(os.path.abspath(__file__))
# Pengaturan lokasi file CSV
def lokasi_csv(nama_file):
    lokasi_program = os.path.join(FOLDER_PROGRAM, nama_file)
    lokasi_terminal = os.path.abspath(nama_file)
    if os.path.isfile(lokasi_program):
        return lokasi_program
    if os.path.isfile(lokasi_terminal):
        return lokasi_terminal
    return lokasi_program
FILE_USERS = lokasi_csv("users.csv")
FILE_LAYANAN = lokasi_csv("layanan.csv")
FILE_TRANSAKSI = lokasi_csv("transaksi.csv")
ROLE_PENGGUNA = ("admin", "user")

KOLOM_USERS = [
    "username",
    "password_hash",
    "role",
    "saldo",
    "nama"
]
KOLOM_LAYANAN = [
    "id_layanan",
    "nama_layanan",
    "harga"
]
```

```
KOLOM_TRANSAKSI = [
    "id_transaksi",
    "waktu",
    "username",
    "jenis",
    "id_layanan",
    "nama_layanan",
    "jumlah",
    "nominal",
    "saldo_awal",
    "saldo_akhir",
    "status"
]

def bersihkan_layar():
    if os.name == "nt":
        os.system("cls")
    else:
        os.system("clear")

def jeda():
    while True:
        try:
            input("\nTekan enter untuk melanjutkan...")
            return
        except KeyboardInterrupt:
            print("\nDimohon untuk tidak CTRL + C ⚠")
        except EOFError:
            print("\nDimohon untuk tidak CTRL + Z ⚠")

# Fungsi untuk input dan validasi
def input_teks(pesan):
    while True:
        try:
            nilai = input(pesan).strip()
            if nilai:
                return nilai
            print("Input tidak boleh kosong. Silakan isi terlebih
dahulu ⚠")
        except KeyboardInterrupt:
            print("\nDimohon untuk tidak CTRL + C ⚠")
        except EOFError:
            print("\nDimohon untuk tidak CTRL + Z ⚠")
```

```
        except (ValueError, TypeError):
            print("⚠ Input tidak valid. Silakan coba lagi. ⚠")

def input_password(pesan):
    while True:
        try:
            password = pwinput(pesan)
            if password.strip():
                return password
            print("⚠ Password tidak boleh kosong. ⚠")
        except KeyboardInterrupt:
            print("\nDimohon untuk tidak CTRL + C ⚠")
        except EOFError:
            print("\nDimohon untuk tidak CTRL + Z ⚠")
        except (ValueError, TypeError):
            print("⚠ Input password tidak valid. Silakan coba
lagi. ⚠")

def input_angka(pesan, minimum=None):
    while True:
        try:
            angka = int(input_teks(pesan))
            if minimum is not None and angka < minimum:
                print("⚠  Input minimal adalah", minimum, "⚠")
            else:
                return angka
        except ValueError:
            print("Input harus berupa angka bulat ⚠")

def konfirmasi(pesan):
    while True:
        pilihan = input_teks(pesan + " (y/n): ").lower()
        if pilihan == "y":
            return True
        if pilihan == "n":
            return False
        print("Masukkan y atau n ⚠")

def format_rupiah(nominal):
    return "Rp" + format(int(nominal), ",").replace(",", ".")

# Tampilan judul dan tabel
def tampilkan_judul(teks, ikon="✨", lebar=44):
```

```
    isi = f"{ikon} {teks} {ikon}"
    ruang = lebar - 4
    if len(isi) > ruang:
        ruang = len(isi)
        lebar = ruang + 4
    kiri = (ruang - len(isi)) // 2
    kanan = ruang - len(isi) - kiri
    print("┌" + "─" * (lebar - 2) + "┐")
    print("│ " + " " * kiri + isi + " " * kanan + "")
    print("└" + "─" * (lebar - 2) + "┘")

def tampilkan_tabel(judul, kolom, data):
    tabel = PrettyTable()
    tabel.title = judul
    tabel.field_names = kolom
    tabel.align = {nama_kolom: "l" for nama_kolom in kolom}
    tabel.header = True
    tabel.hrules = 1
    for indeks, nama_kolom in enumerate(kolom):
        if "harga" in nama_kolom.lower() or "nominal" in
nama_kolom.lower() or "saldo" in nama_kolom.lower() or "total" in
nama_kolom.lower():
            tabel.align[nama_kolom] = "r"
    for baris in data:
        tabel.add_row(baris)
    print(tabel)

# Membaca dan menyimpan data CSV
def baca_csv(lokasi, kolom):
    try:
        with open(lokasi, "r", newline="", encoding="utf-8-sig")
as file:
            pembaca = csv.DictReader(file)
            if pembaca.fieldnames != kolom:
                raise ValueError(
                    "Format kolom file tidak sesuai: " + lokasi
                )
            data = []
            for baris in pembaca:
                if None in baris or any(baris.get(kolom_csv) is
None for kolom_csv in kolom):
                    raise ValueError(
```

```
                        "Terdapat data dengan format tidak
sesuai: " + lokasi
                    )
                data.append(baris)
            return data
    except FileNotFoundError:
        return []

def simpan_csv(lokasi, kolom, data):
    with open(lokasi, "w", newline="", encoding="utf-8") as file:
        penulis = csv.DictWriter(file, fieldnames=kolom)
        penulis.writeheader()
        penulis.writerows(data)

def baca_csv_layanan():
    try:
        with open(FILE_LAYANAN, "r", newline="", encoding="utf-8-
sig") as file:
            pembaca = csv.DictReader(file)
            kolom_file = pembaca.fieldnames or []
            if not all(kolom in kolom_file for kolom in
("id_layanan", "nama_layanan", "harga")):
                raise ValueError("Format kolom file layanan.csv
tidak sesuai.")
            data_layanan = []
            for baris in pembaca:
                data_layanan.append({
                    "id_layanan": (baris.get("id_layanan") or
"").strip(),
                    "nama_layanan": (baris.get("nama_layanan") or
"").strip(),
                    "harga": (baris.get("harga") or "").strip()
                })
        return data_layanan
    except FileNotFoundError:
        return []

def simpan_csv_layanan(data_layanan):
    simpan_csv(FILE_LAYANAN, KOLOM_LAYANAN, data_layanan)

def kunci_id_layanan(layanan):
    return layanan["id_layanan"].upper()
```

```
# Pengolahan data akun dan data awal
def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

def inisialisasi_data():
    if not os.path.exists(FILE_USERS):
        data_users = [
            {
                "username": "admin",
                "password_hash": hash_password("Adminklinik2"),
                "role": "admin",
                "saldo": "0",
                "nama": "Administrator Klinik"
            }
        ]
        simpan_csv(FILE_USERS, KOLOM_USERS, data_users)
    else:
        data_users = baca_csv(FILE_USERS, KOLOM_USERS)
        if cari_user("admin", data_users) is None:
            data_users.append({
                "username": "admin",
                "password_hash": hash_password("Adminklinik2"),
                "role": "admin",
                "saldo": "0",
                "nama": "Administrator Klinik"
            })
            simpan_csv(FILE_USERS, KOLOM_USERS, data_users)
    data_layanan_awal = [
        {"id_layanan": "L001", "nama_layanan": "Konsultasi Umum",
"harga": "50000"},
        {"id_layanan": "L002", "nama_layanan": "Pemeriksaan
Tekanan Darah", "harga": "20000"},
        {"id_layanan": "L003", "nama_layanan": "Pemeriksaan Gula
Darah", "harga": "35000"},
        {"id_layanan": "L004", "nama_layanan": "Pemeriksaan
Kolesterol", "harga": "45000"},
        {"id_layanan": "L005", "nama_layanan": "Konsultasi
Kesehatan", "harga": "40000"}
    ]
    perlu_buat_layanan = not os.path.exists(FILE_LAYANAN)
    if not perlu_buat_layanan:
        try:
            data_layanan_tersimpan = baca_csv_layanan()
```

```
            if not data_layanan_tersimpan:
                perlu_buat_layanan = True
            else:
                data_layanan_tersimpan.sort(key=kunci_id_layanan)
                simpan_csv_layanan(data_layanan_tersimpan)
        except (ValueError, csv.Error, OSError):
            perlu_buat_layanan = True
    if perlu_buat_layanan:
        simpan_csv_layanan(data_layanan_awal)

def cari_user(username, data_users):
    for user in data_users:
        if user["username"].lower() == username.lower():
            return user
    return None

def cari_layanan(id_layanan, data_layanan):
    for layanan in data_layanan:
        if layanan["id_layanan"].lower() == id_layanan.lower():
            return layanan
    return None

# Registrasi dan login pengguna
def registrasi():
    bersihkan_layar()
    tampilkan_judul("Registrasi Pengguna", "📝")
    data_users = baca_csv(FILE_USERS, KOLOM_USERS)
    while True:
        username = input_teks("Buat username: ")
        if len(username) < 4:
            print("Username minimal 4 karakter!")
        elif not username.isalnum():
            print("Username hanya boleh menggunakan huruf dan
angka!")
        elif cari_user(username, data_users) is not None:
            print("Username sudah digunakan!")
        else:
            break
    while True:
        password = input_password("Buat password (minimal 8
karakter): ")
        if len(password) < 8:
            print("Password minimal 8 karakter.")
```

```
        else:
            password_konfirmasi = input_password("Konfirmasi
password: ")
            if password != password_konfirmasi:
                print("Konfirmasi password tidak sesuai.")
            else:
                break
    while True:
        nama = input_teks("Nama lengkap: ")
        if nama == "":
            print("Nama lengkap tidak boleh kosong!")
        else:
            break
    user_baru = {
        "username": username,
        "password_hash": hash_password(password),
        "role": "user",
        "saldo": "0",
        "nama": nama
    }
    data_users.append(user_baru)
    try:
        simpan_csv(FILE_USERS, KOLOM_USERS, data_users)
        print("\n☑ Registrasi berhasil! ☑")
        print("Silakan login menggunakan akun yang telah
dibuat.")
        jeda()
    except OSError:
        data_users.remove(user_baru)
        print("Registrasi gagal karena data tidak dapat
disimpan.")
        jeda()

def login():
    data_users = baca_csv(FILE_USERS, KOLOM_USERS)
    tampilkan_judul("Login", "🔐")
    percobaan = 0
    while percobaan < 3:
        username = input_teks("Username: ")
        password = input_password("Password: ")
        user = cari_user(username, data_users)
        if user is not None:
            if user["password_hash"] == hash_password(password):
```

```
                if user["role"] not in ROLE_PENGGUNA:
                    print("Role akun tidak valid.")
                    jeda()
                    return None
                try:
                    int(user["saldo"])
                except (ValueError, TypeError):
                    print("Saldo akun tidak valid. Periksa data
pengguna.")
                    jeda()
                    return None
                print("\n☑ Login berhasil. ☑")
                print("\n👋 Selamat datang,", user["nama"],
"👋")
                time.sleep(1)
                return user
        percobaan += 1
        sisa = 3 - percobaan
        print("❌ Username atau password salah. ❌")
        if sisa > 0:
            print("Sisa percobaan:", sisa)
    print("\nBatas percobaan login telah habis!")
    jeda()
    return None

# Pengelolaan daftar layanan klinik
def tampilkan_layanan():
    data_layanan = baca_csv_layanan()
    data_layanan.sort(key=kunci_id_layanan)
    isi_tabel = []
    for layanan in data_layanan:
        isi_tabel.append([
            layanan["id_layanan"],
            layanan["nama_layanan"],
            format_rupiah(layanan["harga"])
        ])
    if not isi_tabel:
        print("Belum ada layanan yang tersedia ⚠")
        print("Silakan tambahkan layanan terlebih dahulu.")
    else:
        tampilkan_tabel(
            "✦  DAFTAR LAYANAN KLINIK  ✦",
            ["ID Layanan", "Nama Layanan", "Harga (Rp)"],
```

```
            isi_tabel
        )

def tambah_layanan():
    bersihkan_layar()
    tampilkan_judul("Tambah Layanan", "➕")
    data_layanan = baca_csv_layanan()
    while True:
        id_layanan = input_teks("ID layanan baru (L001):
").upper()
        if len(id_layanan) != 4 or not
id_layanan.startswith("L00") or not id_layanan[3].isdigit():
            print("ID harus 4 karakter dengan format L00X ⚠️")
        elif cari_layanan(id_layanan, data_layanan) is not None:
            print("ID layanan sudah digunakan ⚠️")
        else:
            break
    while True:
        nama_layanan = input_teks("Nama layanan: ")
        nama_sudah_ada = any(
            layanan["nama_layanan"].lower() ==
nama_layanan.lower()
            for layanan in data_layanan
        )
        if nama_sudah_ada:
            print("Nama layanan sudah digunakan ⚠️")
        else:
            break
    harga = input_angka("Harga layanan: Rp", 1)
    layanan_baru = {
        "id_layanan": id_layanan,
        "nama_layanan": nama_layanan,
        "harga": str(harga)
    }
    data_layanan.append(layanan_baru)
    data_layanan.sort(key=kunci_id_layanan)
    try:
        simpan_csv_layanan(data_layanan)
        print("\n☑️ Layanan berhasil ditambahkan. ☑️")
    except OSError:
        data_layanan.remove(layanan_baru)
        print("Layanan gagal disimpan.")
    jeda()
```

```
def ubah_layanan():
    bersihkan_layar()
    tampilkan_layanan()
    data_layanan = baca_csv_layanan()
    print("\n🖊 Ubah Data Layanan 🖊")
    while True:
        id_layanan = input_teks("Masukkan ID layanan (L001):
").upper()
        if len(id_layanan) != 4 or not
id_layanan.startswith("L00") or not id_layanan[3].isdigit():
            print("ID harus 4 karakter dengan format L00X ⚠")
        else:
            break
    layanan = cari_layanan(id_layanan, data_layanan)
    if layanan is None:
        print("Layanan tidak ditemukan.")
        jeda()
        return
    print("\nData layanan yang dipilih:")
    print("Nama:", layanan["nama_layanan"])
    print("Harga:", format_rupiah(layanan["harga"]))
    nama_lama = layanan["nama_layanan"]
    harga_lama = layanan["harga"]
    while True:
        nama_baru = input_teks("Nama layanan baru: ")
        nama_duplikat = any(
            item["id_layanan"].lower() !=
layanan["id_layanan"].lower()
            and item["nama_layanan"].lower() == nama_baru.lower()
            for item in data_layanan
        )
        if nama_duplikat:
            print("Nama layanan sudah digunakan ⚠")
        else:
            break
    layanan["nama_layanan"] = nama_baru
    layanan["harga"] = str(input_angka("Harga baru: Rp", 1))
    try:
        simpan_csv_layanan(data_layanan)
        print("\n☑ Data layanan berhasil diubah. ☑")
    except OSError:
        layanan["nama_layanan"] = nama_lama
```

```
        layanan["harga"] = harga_lama
        print("Data layanan gagal disimpan. Perubahan
dibatalkan.")
    jeda()

def hapus_layanan():
    bersihkan_layar()
    tampilkan_layanan()
    data_layanan = baca_csv_layanan()
    print("\n✦ Hapus Layanan ✦")
    id_layanan = input_teks("Masukkan ID layanan: ")
    layanan = cari_layanan(id_layanan, data_layanan)
    if layanan is None:
        print("Layanan tidak ditemukan.")
        jeda()
        return
    if not konfirmasi("Yakin ingin menghapus layanan " +
layanan["nama_layanan"] + "?"):
        print("Penghapusan dibatalkan.")
        jeda()
        return
    indeks = data_layanan.index(layanan)
    data_layanan.remove(layanan)
    try:
        simpan_csv_layanan(data_layanan)
        print("Layanan berhasil dihapus dari daftar.")
    except (OSError, ValueError, csv.Error) as error:
        data_layanan.insert(indeks, layanan)
        print("Layanan gagal dihapus:", error)
    jeda()

def menu_kelola_layanan():
    while True:
        bersihkan_layar()
        tampilkan_judul("Kelola Layanan Klinik", "🩺")
        print("1. Lihat layanan 🖨️")
        print("2. Tambah layanan ➕")
        print("3. Ubah layanan 🖊️")
        print("4. Hapus layanan 🗑️")
        print("0. Kembali ↩️")
        pilihan = input_teks("\nPilih menu: ")
        if pilihan == "1":
            bersihkan_layar()
```

```
            tampilkan_layanan()
            jeda()
        elif pilihan == "2":
            tambah_layanan()
        elif pilihan == "3":
            ubah_layanan()
        elif pilihan == "4":
            hapus_layanan()
        elif pilihan == "0":
            break
        else:
            print("Pilihan tidak tersedia ⚠")
            jeda()

# Pencatatan transaksi dan invoice
def buat_id_transaksi():
    data = baca_csv(FILE_TRANSAKSI, KOLOM_TRANSAKSI)
    tanggal = datetime.now().strftime("%Y%m%d")
    id_terpakai = [transaksi["id_transaksi"] for transaksi in
data]

    while True:
        nomor = random.randint(1000, 9999)
        id_transaksi = "TRX" + tanggal + str(nomor)
        if id_transaksi not in id_terpakai:
            return id_transaksi

def simpan_transaksi(username, jenis, id_layanan="",
nama_layanan="", jumlah=1,
                     nominal=0, saldo_awal=0, saldo_akhir=0,
status="Berhasil"):
    data = baca_csv(FILE_TRANSAKSI, KOLOM_TRANSAKSI)
    data.append({
        "id_transaksi": buat_id_transaksi(),
        "waktu": datetime.now().strftime("%d-%m-%Y %H:%M:%S"),
        "username": username,
        "jenis": jenis,
        "id_layanan": id_layanan,
        "nama_layanan": nama_layanan,
        "jumlah": str(jumlah),
        "nominal": str(nominal),
        "saldo_awal": str(saldo_awal),
        "saldo_akhir": str(saldo_akhir),
```

```
        "status": status
    })
    simpan_csv(FILE_TRANSAKSI, KOLOM_TRANSAKSI, data)
    return data[-1]

def tampilkan_invoice(transaksi):
    garis = "—" * 44
    waktu_transaksi = transaksi.get("waktu") or
datetime.now().strftime("%d-%m-%Y %H:%M:%S")
    jenis = transaksi.get("jenis", "-").replace("_", " ").title()
    status = transaksi.get("status", "Tidak diketahui")
    ikon_status = "☑" if status.lower() == "berhasil" else "⚠"
    tampilkan_judul("INVOICE KLINIK", "🖨", 44)
    print()
    print("📌 INFORMASI TRANSAKSI")
    print(garis)
    print(f"🆔 No. Transaksi : {transaksi.get('id_transaksi', '-
')}")
    print(f"🧮 Tanggal       : {waktu_transaksi}")
    print(f"👤 Pengguna      : {transaksi.get('username', '-
')}")
    print(f"🗂  Jenis         : {jenis}")
    if transaksi.get("nama_layanan"):
        print()
        print("🖨 RINCIAN LAYANAN")
        print(garis)
        print(f"🩹 Layanan       :
{transaksi.get('nama_layanan', '-')}")
        print(f"🔢 Jumlah        : {transaksi.get('jumlah',
'1')}")
    print()
    print("💰 RINCIAN SALDO")
    print(garis)
    print(f"💵 Nominal       :
{format_rupiah(transaksi.get('nominal', 0))}")
    print(f"⬅  Saldo awal    :
{format_rupiah(transaksi.get('saldo_awal', 0))}")
    print(f"➡  Saldo akhir   :
{format_rupiah(transaksi.get('saldo_akhir', 0))}")
    print(f"{ikon_status} Status        : {status}")
    print(garis)
    print("💙 Terima kasih telah menggunakan layanan kami.")
```

```
# Top-up saldo dan pmbyran layanan
def topup_saldo(user):
    bersihkan_layar()
    tampilkan_judul("Top-up Saldo", "💳")
    nominal = input_angka("Nominal top-up (Minimal Rp.1000): ",
1000)
    if nominal > 10000000:
        print("Nominal maksimal top-up adalah Rp10.000.000.")
        jeda()
        return
    if not konfirmasi("Konfirmasi top-up " +
format_rupiah(nominal) + "?"):
        print("Top-up dibatalkan.")
        jeda()
        return
    data_users = baca_csv(FILE_USERS, KOLOM_USERS)
    tersimpan = cari_user(user["username"], data_users)
    if tersimpan is None:
        print("Akun tidak ditemukan.")
        jeda()
        return
    saldo_awal = int(tersimpan["saldo"])
    saldo_akhir = saldo_awal + nominal
    tersimpan["saldo"] = str(saldo_akhir)
    try:
        simpan_csv(FILE_USERS, KOLOM_USERS, data_users)
        transaksi = simpan_transaksi(user["username"], "TOP_UP",
nominal=nominal,
                                     saldo_awal=saldo_awal,
saldo_akhir=saldo_akhir)
        user["saldo"] = str(saldo_akhir)
        print("\n┌──────────────────────────────────────────────┐")
        print("│            ☑ TOP-UP BERHASIL ☑              │")
        print("└──────────────────────────────────────────────┘")
        tampilkan_invoice(transaksi)
    except (OSError, ValueError, csv.Error) as error:
        tersimpan["saldo"] = str(saldo_awal)
        try:
            simpan_csv(FILE_USERS, KOLOM_USERS, data_users)
        except OSError:
            pass
        print("❌ Top-up gagal disimpan:", error)
    jeda()
```

```
def bayar_layanan(user):
    bersihkan_layar()
    tampilkan_judul("Pembayaran Layanan", "🖨️")
    tampilkan_layanan()
    data_layanan = baca_csv_layanan()
    if not data_layanan:
        print("Belum ada layanan yang terdaftar.")
        jeda()
        return
    layanan = cari_layanan(input_teks("\nMasukkan ID layanan: "),
data_layanan)
    if layanan is None:
        print("ID layanan tidak ditemukan ⚠️")
        jeda()
        return
    jumlah = 1
    total = int(layanan["harga"])
    data_users = baca_csv(FILE_USERS, KOLOM_USERS)
    tersimpan = cari_user(user["username"], data_users)
    if tersimpan is None:
        print("Akun tidak ditemukan.")
        jeda()
        return
    saldo_awal = int(tersimpan["saldo"])
    if saldo_awal < total:
        print("❌ Saldo tidak mencukupi.")
        print("Total pembayaran:", format_rupiah(total))
        print("Saldo saat ini  :", format_rupiah(saldo_awal))
        jeda()
        return
    tampilkan_judul("Ringkasan Pembayaran", "🧾")
    tampilkan_tabel("🧾 Rincian Tagihan 🧾", ["Layanan",
"Harga", "Jumlah", "Total"],
                    [[layanan["nama_layanan"],
format_rupiah(layanan["harga"]),
                      jumlah, format_rupiah(total)]])
    if not konfirmasi("Lanjutkan pembayaran?"):
        print("Pembayaran dibatalkan.")
        jeda()
        return
    saldo_akhir = saldo_awal - total
    tersimpan["saldo"] = str(saldo_akhir)
```

```
    try:
        simpan_csv(FILE_USERS, KOLOM_USERS, data_users)
        transaksi = simpan_transaksi(user["username"],
"PEMBAYARAN_LAYANAN",
                                     layanan["id_layanan"],
layanan["nama_layanan"],
                                     jumlah, total, saldo_awal,
saldo_akhir)
        user["saldo"] = str(saldo_akhir)
        print("\n┌──────────────────────────────────────────────┐")
        print("│          ☑ PEMBAYARAN BERHASIL ☑             │")
        print("└──────────────────────────────────────────────┘")
        tampilkan_invoice(transaksi)
    except (OSError, ValueError, csv.Error) as error:
        tersimpan["saldo"] = str(saldo_awal)
        try:
            simpan_csv(FILE_USERS, KOLOM_USERS, data_users)
        except OSError:
            pass
        print("❌ Pembayaran gagal disimpan:", error)
    jeda()

# Riwayat transaksi dan pengelolaan pengguna
def tampilkan_riwayat(username=None):
    data = baca_csv(FILE_TRANSAKSI, KOLOM_TRANSAKSI)
    if username is not None:
        data = [x for x in data if x["username"].lower() ==
username.lower()]
    data.reverse()
    baris = [[x["id_transaksi"], x["waktu"], x["username"],
            x["jenis"].replace("_", " ").title(),
            x["nama_layanan"] or "-",
format_rupiah(x["nominal"]), x["status"]]
            for x in data]
    tampilkan_tabel("📋 RIWAYAT TRANSAKSI 📋",
                    ["ID Transaksi", "Waktu", "Username",
"Jenis", "Layanan", "Nominal", "Status"],
                    baris if baris else [["-", "-", "-", "-",
"Belum ada transaksi", "-", "-"]])

def menu_kelola_pengguna():
    while True:
        bersihkan_layar()
```

```
        tampilkan_judul("Kelola Pengguna", "👥")
        data = baca_csv(FILE_USERS, KOLOM_USERS)
        tampilkan_tabel("👥 Daftar Pengguna 👥",
                        ["Username", "Nama", "Role", "Saldo"],
                        [[u["username"], u["nama"], u["role"],
format_rupiah(u["saldo"])] for u in data])
        print("\n1. Ubah role pengguna 👥")
        print("2. Hapus pengguna 🗑️")
        print("0. Kembali ↩️")
        pilihan = input_teks("Pilih menu: ")
        if pilihan == "0":
            return
        if pilihan == "1":
            username = input_teks("Username pengguna: ")
            target = cari_user(username, data)
            if target is None:
                print("Pengguna tidak ditemukan.")
            elif target["username"].lower() == "admin":
                print("Role akun admin utama tidak dapat
diubah.")
            else:
                while True:
                    role = input_teks("Role baru (admin/user):
").lower()
                    if role in ROLE_PENGGUNA:
                        break
                    print("Role hanya boleh admin atau user.
Silakan input lagi.")
                if konfirmasi("Ubah role menjadi " + role + "?"):
                    role_lama = target["role"]
                    target["role"] = role
                    try:
                        simpan_csv(FILE_USERS, KOLOM_USERS, data)
                        print("☑️ Role pengguna berhasil
diubah.")
                    except OSError:
                        target["role"] = role_lama
                        print("❌ Perubahan role gagal disimpan.
Data dipulihkan.")
            jeda()
        elif pilihan == "2":
            username = input_teks("Username pengguna yang akan
dihapus: ")
```

```
            target = cari_user(username, data)
            if target is None:
                print("Pengguna tidak ditemukan.")
            elif target["username"].lower() == "admin":
                print("Akun admin utama tidak dapat dihapus.")
            elif konfirmasi("Hapus akun " + target["username"] +
"?"):
                indeks = data.index(target)
                data.remove(target)
                try:
                    simpan_csv(FILE_USERS, KOLOM_USERS, data)
                    print("☑ Pengguna berhasil dihapus.")
                except OSError:
                    data.insert(indeks, target)
                    print("❌ Pengguna gagal dihapus. Data
dipulihkan.")
            jeda()
        else:
            print("Pilihan tidak tersedia ⚠")
            jeda()

# Laporan transaksi tuk admin
def laporan_transaksi():
    data = baca_csv(FILE_TRANSAKSI, KOLOM_TRANSAKSI)
    berhasil = [x for x in data if x["status"] == "Berhasil"]
    total_topup = sum(int(x["nominal"]) for x in berhasil if
x["jenis"] == "TOP_UP")
    total_bayar = sum(int(x["nominal"]) for x in berhasil if
x["jenis"] == "PEMBAYARAN_LAYANAN")
    tampilkan_judul("Laporan Transaksi", "📊")
    tampilkan_tabel("📊 Ringkasan 📊", ["Indikator", "Jumlah"],
                    [["Total transaksi", len(data)],
                    ["Transaksi berhasil", len(berhasil)],
                    ["Total top-up", format_rupiah(total_topup)],
                    ["Total pembayaran layanan",
format_rupiah(total_bayar)]])
    layanan_terjual = {}
    for x in berhasil:
        if x["jenis"] == "PEMBAYARAN_LAYANAN" and
x["nama_layanan"]:
            layanan_terjual[x["nama_layanan"]] =
layanan_terjual.get(x["nama_layanan"], 0) + 1
    if layanan_terjual:
```

```
        terlaris = max(layanan_terjual, key=layanan_terjual.get);
tampilkan_tabel("🏆 Layanan Terlaris 🏆", ["Indikator",
"Jumlah"], [["Nama layanan", terlaris], ["Jumlah transaksi",
layanan_terjual[terlaris]]])
    else:
        print("Belum ada transaksi layanan yang berhasil.")
    tampilkan_riwayat()

# Menu bdasrkan role pengguna
def menu_admin(user):
    while True:
        bersihkan_layar()
        tampilkan_judul("Administrasi Klinik", "🖨️")
        print("Nama pengguna :", user["nama"])
        print("Role          : Admin")
        print("\n1. Kelola layanan 🖨️")
        print("2. Kelola pengguna 👥")
        print("3. Lihat seluruh transaksi 📋")
        print("4. Laporan transaksi 📊")
        print("5. Logout 🔙")
        print("0. Keluar program 🔙")
        pilihan = input_teks("\nPilih menu: ")
        if pilihan == "1":
            menu_kelola_layanan()
        elif pilihan == "2":
            menu_kelola_pengguna()
        elif pilihan == "3":
            bersihkan_layar()
            tampilkan_riwayat()
            jeda()
        elif pilihan == "4":
            bersihkan_layar()
            laporan_transaksi()
            jeda()
        elif pilihan == "5":
            print("👋 Logout berhasil. 👋")
            jeda()
            break
        elif pilihan == "0":
            if konfirmasi("Yakin ingin keluar dari program?"):
                return False
        else:
            print("Pilihan tidak tersedia ⚠️")
```

```
            jeda()
    return True

def menu_user(user):
    while True:
        bersihkan_layar()
        data_users = baca_csv(FILE_USERS, KOLOM_USERS)
        user_tersimpan = cari_user(user["username"], data_users)
        if user_tersimpan is None:
            print("Data pengguna tidak ditemukan.")
            jeda()
            return True
        user["saldo"] = user_tersimpan["saldo"]
        tampilkan_judul("Layanan Klinik", "🩺")
        print("Nama pengguna :", user["nama"])
        print("Username      :", user["username"])
        print("Saldo         :", format_rupiah(user["saldo"]))
        print("\n1. Lihat layanan 🪑")
        print("2. Top-up saldo 💰")
        print("3. Bayar layanan 💳")
        print("4. Lihat riwayat transaksi 🧾")
        print("5. Logout 🚪")
        print("0. Keluar program 🚪")
        pilihan = input_teks("\nPilih menu: ")
        if pilihan == "1":
            bersihkan_layar()
            tampilkan_layanan()
            jeda()
        elif pilihan == "2":
            topup_saldo(user)
        elif pilihan == "3":
            bayar_layanan(user)
        elif pilihan == "4":
            bersihkan_layar()
            tampilkan_riwayat(user["username"])
            jeda()
        elif pilihan == "5":
            print("👋 Logout berhasil. 👋")
            jeda()
            break
        elif pilihan == "0":
            if konfirmasi("Yakin ingin keluar dari program?"):
                return False
```

```
        else:
            print("Pilihan tidak tersedia ⚠")
            jeda()
    return True

# Menu autentikasi dan program utama
def menu_autentikasi():
    while True:
        bersihkan_layar()
        tampilkan_judul("Administrasi Transaksi Klinik", "🖨")
        print("1. Login 🔑")
        print("2. Registrasi pengguna 📝")
        print("0. Keluar program 🪫")
        pilihan = input_teks("\nPilih menu: ")
        if pilihan == "1":
            user = login()
            if user is not None:
                return user
        elif pilihan == "2":
            registrasi()
        elif pilihan == "0":
            if konfirmasi("Yakin ingin keluar dari program?"):
                return None
        else:
            print("Pilihan tidak tersedia ⚠")
            jeda()

def main():
    try:
        inisialisasi_data()
        while True:
            user = menu_autentikasi()
            if user is None:
                bersihkan_layar()
                print(
                    "Terima kasih telah menggunakan "
                    "Administrasi Transaksi Klinik."
                )
                break
            if user["role"] == "admin":
                lanjut = menu_admin(user)
            else:
                lanjut = menu_user(user)
```

```
            if not lanjut:
                bersihkan_layar()
                print(
                    "Terima kasih telah menggunakan "
                    "Administrasi Transaksi Klinik."
                )
                break
    except FileNotFoundError as error:
        print("File tidak ditemukan:", error)
    except ValueError as error:
        print("Terjadi kesalahan pada data:", error)
    except OSError as error:
        print("Terjadi kesalahan saat mengakses file:", error)

if __name__ == "__main__":
    main()
```

BAB IV
PENUTUP

4.1 Kesimpulan

Berdasarkan hasil pembuatan program Administrasi Transaksi Klinik, dapat disimpulkan bahwa program ini dirancang untuk membantu mempermudah proses administrasi dan pencatatan transaksi klinik secara digital. Program ini memiliki beberapa fitur, seperti registrasi dan login pengguna, pembagian hak akses admin dan user, pengelolaan data layanan klinik, top-up saldo, pembayaran layanan, pencatatan riwayat transaksi, serta pembuatan laporan transaksi. Penggunaan bahasa pemrograman Python, library pendukung, dan file CSV sebagai media penyimpanan memungkinkan data pengguna, layanan, dan transaksi dikelola serta disimpan untuk digunakan kembali. Program ini juga menerapkan validasi input untuk mengurangi kesalahan dalam memasukkan data. Dengan adanya program ini, proses administrasi klinik dapat dilakukan secara lebih terstruktur, sehingga dapat menjadi dasar pengembangan sistem administrasi klinik yang lebih efektif dan efisien.

4.2 Saran

Program Administrasi Transaksi Klinik ini masih dapat dikembangkan agar lebih optimal ketika diterapkan dalam kehidupan nyata. Salah satu pengembangannya adalah mengganti penyimpanan file CSV dengan basis data seperti MySQL atau PostgreSQL agar pengelolaan data lebih terstruktur dan mendukung penggunaan oleh banyak pengguna. Selain itu, program dapat dikembangkan menjadi aplikasi berbasis web atau mobile agar lebih mudah diakses oleh admin dan pasien. Aspek keamanan juga perlu ditingkatkan, terutama dalam perlindungan data pribadi pengguna dan pengamanan proses transaksi. Dengan pengembangan tersebut, program diharapkan dapat menjadi sistem administrasi klinik yang lebih lengkap, aman, dan sesuai dengan kebutuhan pelayanan kesehatan.

LAMPIRAN

Lampiran 1. Tabel Kontribusi

Nama	Kontribusi	Bagian
Muhammad Atha Andhika(2609116065)	Coding Flowchart Laporan Github	Memastikan semua anggota berkontribusi dan berkerja sama satu dengan lainnya, merancang alur dan struktur program, membuat programnya serta melakukan pengawasan dan testing, menyusun laporan dan alur flowchart, serta upload file pada repositories.
Nabilah Rahmadhani(2609116041)	Flowchart Laporan README	Pembuatan flowchart, menyusun laporan dan file README berisi penjelasan dan fitur-fitur program
Mario Bawa Welan(2609116054)	Flowchart Laporan README Coding	Membantu pembuatan code dan testing code, menyusun laporan, dan file README bagian output penjelasan menu admin
Annisa Maulida Rizky(2609116061)	Flowchart Laporan README	Membantu pembuatan flowchart, menyusun laporan, dan file README bagian fitur setiap role dan output penjelasan menu user